

Gpyro - A Generalized Pyrolysis Model for Combustible Solids Users' Guide

Version 2.0
May 19, 2025

ACKNOWLEDGMENTS

The financial support of NSF and NASA is gratefully acknowledged. Gpyro development was funded as part of NSF Award 0730556, “Tackling CFD Modeling of Flame Spread on Practical Solid Combustibles” (September 2007-September 2010). Development of early Gpyro versions was funded by the NASA Graduate Student Researcher Program under Grant #NNC-04HA08H, “Piloted Ignition and Flame Spread on Composite Materials in Partial and Normal Gravity” (August 2004 – August 2007).

Esther Kim at WPI and Amanda Dodd at UC Berkeley have played important roles in model development by serving as beta testers and exercising Gpyro in ways that were not contemplated during its initial development. I would like to thank them for their patience, their bug reports, and their suggestions as to how to improve Gpyro. Bryan Klein at Thunderhead Engineering helped get the source code and other files under Subversion control on Google Code (before the repository was moved to a Reax Engineering server). Thomas Steinhaus from the University of Edinburgh showed great patience as we worked together to get Gpyro running on a cluster there.

Others that have contributed to the development of Gpyro over the years include Nicolas Bal, Bangalore Yashwanth, Lucie Hasalova, Davood Zeinali, Xinyan Huang, Salah Benkorichi, and probably many others that I am forgetting!

CONTENTS

Acknowledgments.....	i
Contents	ii
List of Tables	v
1.0 Introduction.....	1
1.1. What is Gpyro?	1
1.2. Philosophy behind Gpyro	2
1.3. Types of problems that can be analyzed with Gpyro.....	2
1.4. How to use this guide.....	3
2.0 Getting started	4
2.1. Downloading Gpyro.....	4
2.2. Compiling Gpyro	4
2.2.1. Build Procedure (Linux and macOS).....	4
2.2.2. Windows	5
2.2.3. Requirements	5
2.2.4. Adding Gpyro to Your PATH	5
2.3. Running Gpyro (standalone simulation).....	5
2.4. Running Gpyro (property estimation program)	6
3.0 format of Input files	7
4.0 Governing Equations and Model Activation	8
4.1. Mass and Species Conservation.....	8
4.2. Energy Conservation.....	9
4.3. Gas Transport.....	10
4.4. Gas Species Transport.....	12
4.5. Gas-Phase Energy Conservation.....	12
5.0 Setting up simulations to run	14
5.1. Defining Simulations and Batch Mode.....	14
5.2. 0D Calculations (ATG).....	14
6.0 Building Meshs	15
6.1. Mesh Definition and Dimensionality.....	15
6.2. Cell Size, Stability Constraints, and Solver Direction.....	15
6.3. Subdividing the Mesh into Multiple Regions and Creating Complex Geometries	16
6.4. Defining Initial Conditions	16
6.5. Defining Boundary Conditions	17
6.6. Examples of Geometry Definitions	17
7.0 Initial Conditions	20
7.1. Condensed-phase temperature	20
7.2. Gas-phase temperature.....	20
7.3. Pressure	20
7.4. Condensed species mass fractions	20
7.5. Gas species mass fractions.....	20
7.6. Example	20
8.0 Boundary conditions	22
8.1. Time-dependent boundary condition definition.....	22

8.2.	Thermal boundary conditions (condensed phase).....	22
8.2.1.	Fixed temperature	22
8.2.2.	Imposed heat flux.....	22
8.3.	Thermal boundary conditions (gas phase)	23
8.4.	Gas transport boundary conditions	24
8.4.1.	Instantaneous release boundary condition	24
8.4.2.	Pressure-driven gas flow boundary condition.....	24
8.5.	Gas species transport boundary conditions.....	24
8.6.	Example	25
9.0	Condensed-phase thermophysical properties.....	27
9.1.	Material Definition.....	27
9.2.	Thermal properties	27
9.2.1.	Thermal conductivity	27
9.2.2.	Specific Heat Capacity.....	28
9.2.3.	Bulk density	28
9.2.4.	Radiative Properties	28
9.2.5.	Example	28
9.3.	Transport Properties in Porous Media	29
9.3.1.	Solid density.....	29
9.3.2.	Permeability	30
9.3.3.	Pore diameter	30
10.0	Gas-phase thermophysical properties	31
10.1.	Gaseous species definition.....	31
10.2.	Special oxidative species	31
10.3.	Gas heat capacity	31
10.4.	Gas molecular weight	31
10.5.	Diffusive gas properties	31
10.6.	Gas viscosity	32
10.7.	Example	32
11.0	Condensed-phase reactions.....	33
11.1.	Declaring Condensed-Phase Reactions.....	33
11.1.1.	Declaring a reaction	33
11.1.2.	Special case: non-charring reactions.....	34
11.1.3.	Special case: multiple condensed products.....	34
11.2.	Condensed-Phase Stoichiometry and Deformation	34
11.2.1.	Definition of the Stoichiometric Coefficient	34
11.2.2.	Deformation and Volume Change	34
11.3.	Gaseous stoichiometry: the species-yield matrix GYIELDS.....	35
11.4.	Kinetics of Condensed-Phase Reactions.....	35
11.4.1.	General formulation	35
11.4.2.	Temperature Dependence (Arrhenius Law).....	36
11.4.3.	Kinetic models	36
11.5.	Enthalpy of reaction.....	38
11.5.1.	Constant Heat of Reaction	39
11.5.2.	Conservative Formulation of Heat of Reaction	39
11.6.	Example	40

12.0	Homogeneous gas-phase reactions	43
12.1.	Declaring Gas-Phase Reactions	43
12.2.	Chemical Kinetics	43
12.3.	Heat of Reaction	44
12.4.	Example	44
13.0	Output files.....	45
13.1.	Available quantities to dump	45
13.2.	Point dumps	46
13.2.1.	Configuration	46
13.2.2.	Example	47
13.2.3.	Output File Format.....	48
13.3.	Profile dumps	48
13.3.1.	Configuration	48
13.3.2.	Example	49
13.3.3.	Output File Format.....	50
13.4.	Smokeview visualization	50
13.4.1.	Configuration	51
13.4.2.	Example	51
13.4.3.	Output File Format.....	51
13.5.	Changing the dump interval mid-simulation	51
13.6.	Other Output Files.....	52
13.6.1.	.out File	52
13.6.2.	CPU Timing File.....	52
13.6.3.	Iteration Convergence File.....	52
14.0	Material property estimation.....	53
14.1.	Format of input files for material property estimation program	54
14.2.	Miscellaneous parameters – GA_GenInput worksheet.....	55
14.3.	Fitness metric weightings – GA_phi worksheet	56
14.4.	Variables to optimize – GA_Vars worksheet	57
14.5.	Experimental data – 01, 02, 03, etc. worksheets.....	58
	References	60
	Appendix A. Nomenclature	61
	Appendix B. Namelist group names and available keywords.	63
	Appendix C. Gas properties	68

LIST OF TABLES

Table 1: Description of Gpyro Namelist groups.....	7
Table 2: Orientation index of the boundary faces.....	17
Table 3: Available kinetic models in Gpyro.	37
Table 4: Quantities available for dump.....	45
Table 5: Gpyro property estimation Fortran Namelist groups, associated worksheet in spreadsheet-based front end, and description of Namelist group entries.....	54
Table 6: &GPYRO_GENERAL keywords (general parameters)	63
Table 7: &GPYRO_CASES keywords (cases to run)	63
Table 8: &GPYRO_GEOM keywords (geometry specification)	63
Table 9: &GPYRO_IC keywords (initial condition specification).	64
Table 10: &GPYRO_ALLBC keywords (boundary condition specification).....	64
Table 11: &GPYRO_SPROPS keywords (solid properties).....	64
Table 12: &GPYRO_GPROPS keywords (gas properties).....	65
Table 13: &GPYRO_RXNS keywords (condensed phase reaction properties).....	65
Table 14: &GPYRO_GYIELDS keywords (the species-yield matrix GYIELDS).	66
Table 15: &GPYRO_HGYIELDS keywords (the species-yield matrix HGYIELDS).	66
Table 16: &GPYRO_HGRXNS keywords (gas phase reaction properties).....	66
Table 17: &GPYRO_OUTPUT keywords (output quantities).	66
Table 18: &GA_GENINPUT keywords (property estimation general/miscellaneous).	67

1.0 INTRODUCTION

Gpyro is a generalized pyrolysis model for combustible solids (<https://gitlab.imft.fr/gpyro/gpyro>). This document presents the User Guide for Gpyro V2, a substantially redesigned and extended evolution of the original Gpyro pyrolysis code developed by C. Lautenberger (Gpyro V0.8). Although Gpyro V2 retains the conceptual foundations of the initial model, it represents a major rework of the entire codebase. The structure has been reorganized and rewritten in a clearer, more modular, and more maintainable form, resulting in a significantly more robust and stable implementation. Several new numerical solvers have been introduced, the modelling capabilities have been considerably extended, particularly for fully three-dimensional configurations, and the computational performance has been greatly improved. Depending on the test case, Gpyro V2 is between $4\times$ and $200\times$ faster than the original version.

The development history of the initial Gpyro code can be followed through Lautenberger’s User Guide, Technical Reference, and the PhD dissertation on which the original implementation was based [ref]. These documents establish the theoretical and numerical basis from which Gpyro V2 originates. The present version continues this lineage while introducing a complete redesign of the numerical solvers, extended verification procedures, and updated documentation.

The recent developments of Gpyro V2 are documented in Y. Abdelhafiz’s ongoing PhD thesis (2026), on which much of the current redesign is based, as well as in the new Technical Guide accompanying this release and in the following publications [ref, ref]. This User Guide is specifically adapted to Gpyro V2, and while it primarily reflects the structure and capabilities of this new version, many elements remain applicable to the original Gpyro implementation. It may also serve as a complementary reference for users of the original code, as this document completes and clarifies Lautenberger’s original User Guide, particularly in sections where the earlier documentation was incomplete or only partially drafted.

1.1. What is Gpyro?

Gpyro is open-source, freely available computer software that simulates thermo-chemical processes occurring in thermally stimulated solids (loosely, it is a pyrolysis model). It can be used for 0D (lumped), 1D, 2D, or 3D simulations of thermally stimulated solids, including thermal and thermo-oxidative decomposition of condensed-phase species. Gpyro is coupled to Version 6.2.0 of NIST’s Fire Dynamics Simulator (FDS)¹ [10] where it can be applied as a boundary condition to facilitate coupled modeling of ignition and flame spread. That is, calculations can be conducted either as standalone simulations or in a coupled simulation where Gpyro is applied as a boundary condition in FDS.

An integral part of Gpyro is a material property estimation program that can be used to estimate material properties from laboratory tests such as ThermoGravimetric Analysis (TGA), Differential Scanning Calorimetry (DSC), and conventional flammability tests such as the Cone Calorimeter and the Fire Propagation Apparatus (FPA). The basic property estimation routine was originally limited to genetic algorithm optimization [7] and was later extended to additional methods [8]. The

¹ Gpyro and its coupling to FDS is not supported or endorsed by NIST or VTT in any way.

property estimation routines use Message Passing Interface (MPI) to reduce run times with parallel processing. The property estimation routines are “embarrassingly parallel”, and Gpyro’s property estimation routines have been successfully run on 128+ computational cores with near-linear speedup.

1.2. Philosophy behind Gpyro

Rather than hard-coding a certain level of complexity, Gpyro provides the user with the ability to include as much or as little complexity in a calculation as desired. In its simplest form, Gpyro is a 1D transient heat conduction code for a constant property inert (nonreactive) solid. In its most complicated form, Gpyro can be used to solve 3D conjugate heat transfer, fluid flow/pressure evolution, gaseous species diffusion/convection in a multi-component, heterogeneous, reacting porous medium with temperature-dependent thermophysical properties, in-depth radiation absorption, radiation heat transfer across pores, multi-step heterogeneous decomposition kinetics of arbitrary order or complex reaction model, and multi-step homogeneous gas-phase reactions. The level of complexity included in a simulation, and the requisite number of model input parameters/material properties, is decided entirely by the user.

Gpyro development efforts have focused on providing generalized software containing physical phenomena that can be enabled or disabled as desired by the user. Similarly, the material property estimation routines require the user to specify valid parameter ranges as well as other variables/flags that affect how the material property estimation process occurs. Gpyro, and its associated material property estimation routines, are merely tools that can be used to determine which physics matter under which circumstances. Equally as important, Gpyro can be used to develop “validated” sets of material properties that can be used to model the pyrolysis behavior of solid materials under fire conditions.

1.3. Types of problems that can be analyzed with Gpyro

Much effort has been devoted to keeping Gpyro as general as possible to expand the types of problems that can be analyzed. The following is a brief partial list of the types of problems that Gpyro has been used to simulate at some point during its development:

- TGA experiments (primarily for estimating reaction kinetics)
- DSC experiments (primarily for estimating heats of reaction)
- Cone Calorimeter and FPA experiments (primarily for estimating reaction kinetics, heats of reaction, and thermophysical properties)
- Thermal pyrolysis and surface regression of noncharring thermoplastics
- Thermal pyrolysis of charring solids
- Thermal pyrolysis of heterogeneous fiber reinforced polymer composites
- Heating, gasification, swelling of intumescent coatings, and the thermal resistance offered by such coatings
- Effect of ambient oxygen concentration on thermo-oxidative pyrolysis of charring solids
- Smolder wave propagation in porous media
- Transition from smolder to flame in porous media
- Propellant cookoff in a confined housing

- Self-heating and spontaneous ignition of rags soaked in linseed oil
- Ignition of fuel beds by embers, firebrands, and sparks
- Laminar opposed flow flame spread (coupled to FDS)

Other problems that could be simulated with Gpyro either “out of the box” or with minor modifications include:

- Concrete spalling
- Ablation of sacrificial protective coatings
- Behavior of thermally stimulated anisotropic solids (wood, fiber reinforced polymer composites)
- Biomass pyrolysis in fast/flash pyrolysis reactors

1.4. How to use this guide

The best way to use this guide is not necessarily to read it cover to cover. It is recommended that new users run the sample cases included with the Gpyro distribution and then read the sections of this guide as necessary to understand these samples.

2.0 GETTING STARTED

Gpyro is a console application that is executed from a Windows command prompt or Linux console. This chapter describes how to download, compile, and run Gpyro under Windows, Linux and MacOS.

2.1. Downloading Gpyro

Gpyro exists in two versions. The original version, developed and published in the early 2010s, is available on GitHub at <https://github.com/lauteberger/gpyro>. Although this version is no longer actively maintained, it remains a valuable reference for understanding the initial structure and functionality of the code.

In parallel, a maintained fork of Gpyro has been developed to improve performance, extend capabilities, and ensure compatibility with modern systems. This documentation is based on that maintained version available in GitLab at <https://gitlab.imft.fr/gpyro/gpyro>. While the input file structure remains largely consistent between the two versions; some newly introduced options may affect compatibility with the legacy version.

The maintained repository includes the source code for Gpyro Gpyro_propest, and coupled Gpyro/FDS, validation cases, compilation instructions, and several documentation resources: a user guide, a technical guide, and a verification guide.

2.2. Compiling Gpyro

The `build` directory in the Gpyro repository contains all necessary Makefiles to compile the executables `gpyro` and `gpyro_propest` (the Gpyro property estimation program) and `fds6_gpyro` (The coupling between Gpyro and FDS 6.2.0). Compilation is supported on Linux, macOS, and Windows (via provided Makefiles), using either GNU or Intel Fortran compilers. Both release and debug builds are available.

2.2.1. Build Procedure (Linux and macOS)

Two build scripts are provided in `build/linux`:

- `make_gnu.sh`: Compiles both release and debug versions of `gpyro`, `fds6_gpyro` and `gpyro_propest` using the GNU Fortran compiler.
- `make_intel.sh`: Compiles the same versions using the Intel Fortran compiler.

These scripts automatically invoke the appropriate Makefiles and compile all variants in a single run.

Example (GNU Fortran):

```
cd build/linux
bash make_gcc.sh    # Compile with GCC
```

Note that if the compilation fails, it may be due to incorrect compiler paths or environment variables in the script. You can edit the script to match your system's compiler installation.

Compiled executables will be located in:

```
build/linux/gpyro/  
build/linux/gpyro_propest/
```

2.2.2. Windows

Make instructions are available for Windows, but they have not been tested by the developer. It is recommended to use Windows Subsystem for Linux (WSL) and follow the Linux build procedure.

2.2.3. Requirements

Fortran Compiler: GCC $\geq$ 8.1 or Intel Fortran (we use gcc 9.1).

MPI: A working MPI installation is required to compile gpyro_propest (we use OpenMPI 3.1.3).

If you plan to conduct 2D or 3D simulations, then you should also install NIST's Smokeview because Smokeview is used to visualize 2D Gpyro simulations. Smokeview can be downloaded from <https://pages.nist.gov/fds-smv/downloads.html>.

2.2.4. Adding Gpyro to Your PATH

To run gpyro or gpyro_propest from any location, add their build directory to your system's PATH environment variable. Example :

```
export PATH=/path/to/gpyro/build/linux:$PATH
```

To make this change permanent, add the line to your shell configuration file (e.g., ~/.bashrc).

2.3. Running Gpyro (standalone simulation)

Gpyro is executed from the Windows command, or the Linux shell as follows:

```
gpyro input_file.data
```

where input_file.data is a text file that specifies the geometry, boundary conditions, thermophysical properties, reaction kinetics, etc. See Section 3.0 for an explanation of input files. The format of this input file is described in Section 3.0

To run Gpyro in parallel using OpenMP, set the environment variable OMP_NUM_THREADS to the desired number of threads before executing the program. For example, on Linux or macOS:

```
OMP_NUM_THREADS=4  
gpyro input_file.data
```

and on Windows PowerShell:

```
$env:OMP_NUM_THREADS=4
```

```
.\gpyro.exe input_file.data
```

It is generally recommended to use no more than one processor per 500 computational cells, as using more threads may not improve performance and can even degrade it due to communication overhead.

2.4. Running Gpyro (property estimation program)

The Gpyro property estimation algorithm is a separate executable file from the standalone Gpyro executable. It is run in a way that is similar to the standalone executable, but no input file name is specified:

```
gpyro_propest
```

The reason that no input file is specified is that when running the Gpyro property estimation program, the input file name is assumed to be `gpyro.data`. The filename is hardcoded to simplify running in parallel with Message Passing Interface (MPI). See Section 3.0 for an explanation of input files. As described in Section 3.0, when running the property estimation algorithm, it is also necessary to provide (in the working directory) comma separated value files that contain experimental data from which model input parameters will be estimated (unless synthetic experimental data are used).

The Gpyro property estimation program can be run in serial or in parallel, but MPI must be installed even for a serial run or the executable will not run. The above command would launch a serial run of the Gpyro property estimation program. The procedure for starting a parallel run depends on your operating system and the installed distribution and version of MPI. The following command would launch a parallel run on 16 processors under linux:

```
mpirun -np 16 -machinefile ./machines.linux /usr/local/bin/gpyro_propest
```

The syntax may be slightly different for different MPI distributions or under Windows.

3.0 FORMAT OF INPUT FILES

Gpyro reads plain ASCII text files that consist of a sequence of Fortran namelist groups (*e.g.*, &GPYRO_GENERAL) and entries associated with each Namelist group (*e.g.*, P0 = 101325.). The available Namelist groups and entries for each Namelist group are given in Appendix A.

The Namelist groups specific to standalone simulations are listed in Table 1. Those Namelist groups specific to the property estimation algorithm are discussed in Section 14.1.

Table 1: Description of Gpyro Namelist groups

Namelist group name	Description	Reference
&GPYRO_GENERAL	General parameters.	Chapter 4.0 and Table 6
&GPYRO_OUTPUT	output parameters.	Chapter 13.0 Table 3 and Table 17
&GPYRO_SPROPS	condensed phase properties.	Chapter 9.0 and Table 11
&GPYRO_GPROPS	Gas phase properties.	Chapter 10.0 and Table 12
&GPYRO_RXNS	Condensed phase reactions.	Chapter 11.0 and Table 13
&GPYRO_GYIELDS	Condensed phase reactions gaseous yields matrix.	Chapter 11.0 and Table 14
&GPYRO_HGRXNS	gaseous phase reactions.	Chapter 12.0 and Table 16
&GPYRO_HGYIELDS	Gas phase reactions gaseous yields matrix.	Chapter 12.0 and Table 15
&GPYRO_IC	Initial conditions.	Chapter 7.0 and Table 9
&GPYRO_GEOM	Geometry.	Chapter 6.0 and Table 8
&GPYRO_ALLBC	Boundary conditions.	Chapter 8.0 and Table 10
&GPYRO_CASES	Cases to run.	Chapter 5.0 and Table 7

Standard Fortran Namelist group conventions apply. The first character of a Namelist group must be an '&', and the last character must be a '/'. An input file consists of a series of Namelist groups. The order in which the Namelist groups appear in the input file does not matter, but a Namelist group may appear only once in an input file. For example, the following is valid:

```
&GPYRO_GENERAL TAMB          = 293.
                        P0      = 101325. /
```

But the following is not valid:

```
&GPYRO_GENERAL TAMB          = 293. /
&GPYRO_GENERAL P0            = 101325. /
```

4.0 GOVERNING EQUATIONS AND MODEL ACTIVATION

Gpyro solves a coupled set of conservation equations describing heat transfer, condensed-phase decomposition, gas generation, and transport phenomena within thermally degrading porous materials. Depending on the selected modeling options, the solver may additionally account for internal gas pressure evolution, and gas-species transport.

The present chapter describes the governing equations implemented in Gpyro together with the associated modeling assumptions and activation parameters. For each physical model, the corresponding input options required to enable or configure the formulation are identified. Unless otherwise specified, model activation parameters are defined in the `&GPYRO_GENERAL` namelist group.

4.1 Mass and Species Conservation

Gpyro adopts a Lagrangian formulation in which the control volume ΔV of each computational cell evolves in time to follow the amount of condensed-phase material present in the cell. As decomposition proceeds and mass is lost due to gas generation, the volume adjusts accordingly, ensuring consistency between mass and geometry.

The total mass conservation over a control volume ΔV is written as:

$$\frac{dm}{dt} = \frac{d(\bar{\rho} \Delta V)}{dt} = -\dot{\omega}_{fg}''' \Delta V \quad (1)$$

Where $\bar{\rho}$ denotes the bulk density and $\dot{\omega}_{fg}'''$ is the volumetric gas generation rate resulting from chemical reactions.

For each condensed-phase species A , a conservation equation is formulated as:

$$\frac{dm_A}{dt} = \frac{d(Y_A \bar{\rho} \Delta V)}{dt} = \dot{\omega}_{net,A}''' \Delta V \quad (2)$$

where Y_A is the mass fraction of species A , and $\dot{\omega}_{net,A}'''$ is the net rate of production of that species. The net production rate is defined as the difference between formation and destruction rates:

$$\dot{\omega}_{net,A}''' = \dot{\omega}_{f,A}''' - \dot{\omega}_{d,A}''' \quad (3)$$

The formation and destruction terms are obtained by summing the contributions of all K condensed phase reactions (See section 11.0):

$$\dot{\omega}_{d,A}''' = \sum_{k=1}^K \dot{\omega}_{dA,k}''' \quad ; \quad \dot{\omega}_{f,A}''' = \sum_{k=1}^K \dot{\omega}_{fA,k}''' \quad (4)$$

For a given reaction k , the destruction rate $\dot{\omega}_{dA,k}'''$ is equal to zero if the reaction does not consume species A , and equal to the reaction rate r_k if it does. Similarly, the formation rate $\dot{\omega}_{fA,k}'''$ is zero if

species A is not produced by reaction k , and equal to $\theta_k r_k$ if it is, where θ_k denotes the stoichiometric coefficient associated with reaction k .

The total gas formation rate is obtained by summing the contributions of all reactions:

$$\dot{\omega}_{fg}''' = \sum_{k=1}^K \dot{\omega}_{fg,k}''' \quad ; \quad \dot{\omega}_{fg,k}''' = (1 - \theta_k) r_k \quad (5)$$

Additional details regarding reaction mechanisms and associated parameters are provided in Section 11.0.

4.2. Energy Conservation

The conservation of energy is expressed over a control volume (ΔV) as:

$$\frac{d(\bar{h}m)}{dt} = \oint_{\partial V} \vec{k} \cdot \nabla T \cdot \vec{n} dS + \dot{Q}_s''' \Delta V \quad (6)$$

where $\bar{h}$ denotes the specific enthalpy, m the condensed-phase mass, and T the solid temperature, $\vec{k} = (k_x, k_y, k_z)$ the anisotropic thermal conductivity, and $\vec{n}$ the outward normal vector of the volume.

The volumetric source term $\dot{Q}_s'''$ represents the combined effects of radiation absorption, gas–solid heat exchange, chemical reactions, and additional heat transfer mechanisms. It is written as:

$$\dot{Q}_s''' = -\frac{\partial \dot{q}_r''}{\partial z} + \dot{Q}_{s,g}''' + \sum_{A=1}^M h_A \dot{\omega}_{net,A}''' + \sum_{k=1}^K \dot{Q}_k''' + \dot{Q}_{cl}''' \quad (7)$$

Chemical source terms

The term $\sum_{A=1}^M h_A \dot{\omega}_{net,A}'''$ involving the summation over the M codesned species accounts for the variation of sensible enthalpy associated with chemical reactions. In this expression, h_A denotes the sensible enthalpy of species A , while $\dot{\omega}_{net,A}'''$ represents the net rate of production or consumption of that species.

The contribution $\sum_{k=1}^K \dot{Q}_k'''$ corresponds to the heat released or absorbed by chemical reactions. The formulation of this term and the associated input parameters are described in Section 11.5.

Radiation absorption

Radiative heat transfer within the material is modeled using a Beer–Lambert-type attenuation law. The radiative heat flux is expressed as:

$$\dot{q}_r''(z) = \bar{\epsilon} \dot{Q}_{ext} \exp\left(-\int_0^z \bar{\kappa}(z') dz'\right) \quad (8)$$

where $\bar{\epsilon}$ denotes the surface emissivity, $\dot{Q}_{ext}$ is the externally imposed radiative heat flux, and $\bar{\kappa}$ is an effective absorption coefficient.

Gas solid thermal exchange

The gas–solid heat exchange term $\dot{Q}_{s,g}'''$ is controlled through the parameter QSG_MODEL in the GPYRO_GENERAL namelist group. When the option 'NONE' is selected, no thermal interaction is considered, and $\dot{Q}_{s,g}''' = 0$.

When the option 'THERMAL_EQUILIBRIUM' is used, the gas and solid phases are assumed to be locally at the same temperature, and the exchange term is expressed as:

$$\dot{Q}_{s,g}''' = \dot{m}_{gas}'' c_{pg} \frac{dT}{dz} \quad (9)$$

where $\dot{m}_{gas}''$ is the gas mass flux and c_{pg} the gas heat capacity.

QSG_MODEL='THERMAL_EQUILIBRIUM_GENERAL' corresponds to a more general formulation, in which the full gas-phase energy balance is retained (See user guide):

$$\dot{Q}_{s,g}''' = \rho_g \bar{\psi} c_{pg} \frac{\partial T}{\partial t} + \dot{m}'' c_{pg} \frac{\partial T}{\partial z} - \frac{\partial}{\partial z} \left(\bar{\psi} k_g \frac{\partial T}{\partial z} \right) \quad (10)$$

This formulation is included for consistency with earlier versions of the model. However, it is not recommended for general use, as it does not significantly improve predictive capability and has not been fully validated.

An alternative formulation based on a two-temperature approach, in which the gas-phase temperature was explicitly solved, was introduced in Gpyro version 0.8. In that formulation, the gas–solid heat exchange term was expressed as $\dot{Q}_{s,g}''' = h_{cv}(T - T_g)$, where T_g denotes the gas temperature. This approach is no longer available in the current version of Gpyro, as the solver does not explicitly resolve the gas-phase temperature, despite its description in earlier user guides.

volumetric heat loss

In addition, an optional heat loss term may be included to represent convective heat transfer at boundaries that cannot be explicitly resolved in 1D configurations. This term is written as:

$$\dot{Q}_{cl}''' = -h_{lc}(T - T_{amb}) \quad (11)$$

Where h_{lc} is an effective heat transfer coefficient in $\text{W.m}^{-2}.\text{K}^{-1}$ and T_{amb} is the ambient temperature. The parameter h_{lc} is specified through VHLC while T_{amb} is defined via TAMB, both in the GPYRO_GENERAL namelist.

4.3. Gas Transport

Gas transport in Gpyro may be modeled using different levels of approximation, depending on the desired physical fidelity and computational cost. The selected formulation determines how gaseous

products generated by pyrolysis are transported through the porous condensed phase and released at the boundaries.

The simplest approach assumes instantaneous release of gaseous products through the exposed surface. In this formulation, no internal gas accumulation or pressure gradient is resolved within the material. The gas mass flux is directly obtained by integrating the gas generation rate over the remaining material thickness:

$$\dot{m}_{gas}''(z) = \int_z^L \dot{\omega}_{fg}'''(z') \Delta V \quad (12)$$

where $\dot{m}_{gas}''$ is the gas mass flux, $\dot{\omega}_{fg}'''$ is the volumetric gas generation rate resulting from chemical reactions, z is the local depth coordinate, and L is the thickness of the condensed phase. This approximation is computationally efficient and is generally appropriate when internal pressure gradients remain negligible.

A more detailed formulation may be employed when internal gas transport through the porous medium is significant. In that case, gas accumulation and pressure-driven flow are explicitly resolved. The gas-phase mass conservation equation is written as:

$$\frac{dm_g}{dt} = \frac{d\bar{\psi}\rho_g\Delta V}{dt} = \oint_{\partial V} \dot{m}_{gas}'' \cdot \vec{n} dS + \dot{\omega}_{fg}''' \Delta V \quad (13)$$

where $m_g = \bar{\psi}\rho_g$ is the gas mass contained within the control volume, $\bar{\psi}$ is the porosity, ρ_g is the gas density.

The gas mass flux is modeled using Darcy's law:

$$\dot{m}_{gas}'' = \frac{\bar{K}}{\nu} \left(\frac{\partial P}{\partial z} - \mathbf{g}\rho_g \right) \quad (14)$$

where $\bar{K}$ is the permeability, ν is the gas kinematic viscosity, P is the gas pressure, and $\mathbf{g}$ is the gravitational acceleration vector.

The gas density is obtained from the ideal gas law:

$$\rho_g = \frac{PM}{RT_g} \quad (15)$$

Where M is the gas molecular weight, R is the universal gas constant, and T_g is the gas temperature. Combining the continuity equation, Darcy's law, and the ideal gas relation yields the governing pressure equation:

$$\frac{d \left(\bar{\psi} \frac{PM}{RT_g} \Delta V \right)}{dt} = \oint_{\partial V} \frac{\bar{K}}{\nu} \left(\nabla P - P \frac{gM}{RT_g} \right) \cdot \vec{n} dS + \dot{\omega}_{fg}''' \Delta V \quad (16)$$

When this formulation is activated, the gas pressure field is explicitly solved, and the gas mass flux is subsequently computed from Darcy's law.

The pressure solver is activated by setting the parameter `SOLVE_PRESSURE = .TRUE.`. Gravitational acceleration components are specified through the parameters `GX`, `GY`, and `GZ`. When `SOLVE_PRESSURE = .FALSE.`, the instantaneous gas release approximation is used instead, which is the default behavior.

4.4. Gas Species Transport

In addition to the bulk gas transport formulation described previously, Gpyro may also solve a species transport equation for each gas-phase species. This formulation allows the spatial distribution of gaseous species within the porous medium to be resolved and is particularly important for oxidation problems, where oxygen penetration into the material must be explicitly modeled. The transport equation for gas-phase species j is written as:

$$\frac{dY_j \bar{\psi} \rho_g \Delta V}{dt} + \oint_{\partial V} \dot{m}_{gas}'' Y_j \cdot \vec{n} dS = \oint_{\partial V} \bar{\psi} \rho_g D \nabla Y_j \cdot \vec{n} dS + \dot{\omega}_{fg,j}''' \Delta V \quad (17)$$

where Y_j is the gas-phase mass fraction of species j , D is the effective diffusion coefficient, and $\dot{\omega}_{fg,j}'''$ is the volumetric source term associated with the production or consumption of species j . The diffusion coefficient is calculated using the Chapman–Enskog formulation described in Section 10.5.

The first term on the left-hand side represents transient gas species accumulation within the porous medium, while the second term accounts for advective transport driven by the bulk gas flow. The right-hand side includes diffusive transport and chemical source terms.

This equation is solved only when the parameter `SOLVE_YJ = .TRUE.` is specified. By default, gas species transport equations are not solved.

4.5. Gas-Phase Energy Conservation

Earlier versions of the Gpyro documentation presented a formulation in which the conservation of energy in the gas phase could be explicitly solved. In particular, the Gpyro version 0.8 user guide described a gas-phase energy equation and the associated modeling assumptions. However, this capability does not appear to be available in the publicly released source code of Gpyro version 0.8, nor in the current public version of the solver. It is possible that this formulation existed in an internal or unpublished development version of the code.

For completeness, the governing equation originally proposed for gas-phase energy conservation is recalled here:

$$\frac{dh_g \bar{\psi} \rho_g \Delta V}{dt} + \oint_{\partial V} \dot{m}_{gas}'' h_g \cdot \vec{n} dS = \left(\oint_{\partial V} \bar{\psi} \rho_g D \nabla h_g \cdot \vec{n} dS \right)_p + \dot{Q}_g''' \Delta V \quad (18)$$

where h_g denotes the gas-phase specific enthalpy and $\dot{Q}_g'''$ is the volumetric gas-phase energy source term. The gas-phase energy source term is expressed as:

$$\dot{Q}_g''' = -\dot{Q}_{s,g}''' + \sum_{k=1}^K \dot{Q}_{g,k}''' + \sum_{l=1}^L \dot{Q}_{g,l}''' \quad (19)$$

The first term represents gas–solid thermal exchange, while $\dot{Q}_{g,l}'''$ corresponds to the heat release associated with homogeneous gas-phase reactions l . The term $\dot{Q}_{g,k}'''$ represents the fraction of heat generated by condensed-phase reactions k that is transferred directly to the gas phase.

The parameter `SOLVE_GAS_ENERGY=.TRUE.` was originally intended to activate this formulation. However, no operational implementation of the gas-phase energy solver is currently available in the public releases of Gpyro, and the parameter therefore has no effect in the present version of the code.

5.0 SETTING UP SIMULATIONS TO RUN

Gpyro supports simulations in 0D, 1D, 2D, and 3D. The procedures for constructing the mesh geometry for each of these cases are presented in Section 6.0. Once the mesh definitions are available, the simulations to be executed are specified in the namelist group `&GPYRO_CASES`. See Table 7.

A single input file can be used to run several simulations in succession. This is particularly useful for parametric studies in which certain conditions are varied from one case to another. Parameters such as material thickness, applied heat flux, ambient gas composition, front-face heat transfer coefficient, ignition time, flame heat flux, thermophysical properties, and reaction kinetics can be modified between cases. Each simulation is linked to a predefined mesh, ensuring that geometry and boundary conditions are correctly applied.

5.1 Defining Simulations and Batch Mode

The total number of simulations to be executed is given by `NCASES`. For each case `ICASE` (from 1 to `NCASES`), the mesh to be used is specified through `IMESH (ICASE)`, and the simulation end time in seconds is set with `TSTOP (ICASE)`.

The time step used for all simulations is defined in the `&GPYRO_GENERAL` namelist group through the parameter `DT0`.

The example below defines a single simulation lasting 5000 s using mesh 1, which must already have its geometry, initial conditions, and boundary conditions defined elsewhere in the input file:

```
&GPYRO_CASES NCASES = 1
  IMESH(1) = 1 , TSTOP(1) = 5000/
```

5.2 0D Calculations (ATG)

A 0D simulation corresponds to a single homogeneous particle with negligible gradients of temperature ($B_i \ll 1$) and species. This is an idealized model of thermal analysis experiments such as TGA and DSC. Quantities such as particle temperature, mass, and species mass fractions vary temporally but not spatially. In 0D simulations, the particle temperature is specified as a function of time (initial temperature plus a constant linear ramp rate).

For 0D calculations, set `ZEROD(ICASE)` to `.TRUE.` in the `&GPYRO_CASES` group for the desired case. This mode is typically used for TGA or DSC experiments, where the sample is subjected to a constant heating rate. The rate, expressed in °C/min, is set using `BETA (ICASE)`.

The example below specifies two 0D cases: Case 1 runs for 3600 s with a heating rate of 10 °C/min, and Case 2 runs for 1800 s with a heating rate of 20 °C/min.

```
&GPYRO_CASES NCASES      = 2
  IMESH(1) = 1
  ZEROD(1) = .TRUE.
  TSTOP(1) = 3600. , BETA(1) = 10.
  IMESH(2) = 1
  ZEROD(2) = .TRUE.
```

TSTOP(2) = 1800. , BETA(2) = 20./

6.0 BUILDING MESHES

The &GPYRO_GEOM group contains all parameters required to define the geometry (See Table 8). Multiple distinct meshes can be defined, each with its own geometry, initial conditions, and boundary conditions. Each mesh runs independently. Defining multiple meshes is only useful when running in batch mode (See Section 5.1). The total number of meshes is specified by the NMESH variable.

6.1. Mesh Definition and Dimensionality

For a mesh with index IMESH (from 1 to NMESH), the geometry is defined as an orthogonal, regular, 3D rectangular mesh. Its dimensions in the x , y , and z directions (in meters) are specified using the parameters XDIM(IMESH), YDIM(IMESH), and ZDIM(IMESH), respectively. The number of cells in each direction is set through NCELLX(IMESH), NCELLY(IMESH), and NCELLZ(IMESH). The dimensionality of the simulation is configured as follows:

- **0D (ATG)**: NCELLZ(IMESH) = NCELLX(IMESH) = NCELLY(IMESH) = 1, (and see Section 145.2).
- **1D**: NCELLZ(IMESH) > 1 and NCELLX(IMESH) = NCELLY(IMESH) = 1 (refinement only in the z -direction)
- **2D**: NCELLZ(IMESH) > 1, NCELLX(IMESH) > 1 and NCELLY(IMESH) = 1 (refinement in x and z -directions)
- **3D**: NCELLZ(IMESH) > 1, NCELLX(IMESH) > 1, NCELLY(IMESH) > 1

6.2. Cell Size, Stability Constraints, and Solver Direction

To accurately capture pyrolysis phenomena, developer experience indicates that the cell size in the degradation direction should be smaller than 0.1 mm. **In 2D and 3D simulations, particular care must be taken when choosing the number of cells** for two main reasons. First, increasing the number of cells directly increases computational cost, which can rise very quickly in 3D. Second, very small cells in directions solved explicitly by the thermal solver can require prohibitively small time steps to satisfy stability constraints. For an explicitly solved direction (e.g., x), the time step must satisfy the Von Neumann stability criterion :

$$\Delta t < \frac{\Delta x^2}{2\alpha} = \frac{k}{2\rho c_p} \Delta x^2 \quad (20)$$

where α is the thermal diffusivity. This constraint becomes critical when fine cells are used in all directions. For example, with $\Delta x = 0.01 \text{ mm}$ and a typical thermal diffusivity of $\alpha = 10^{-5} \text{ m}^2 \cdot \text{s}^{-1}$, the time step must be smaller than $5 \times 10^{-6} \text{ s}$.

To avoid this limitation, the SOLVER_MAIN_DIRECTIONS option in the &GPYRO_GENERAL namelist allows specifying one direction to be solved implicitly. The stability constraint is then removed for the chosen implicit direction. Since pyrolysis is often dominant along a single axis (typically z) it is recommended to use fine cells in z while keeping coarser cells (e.g., $\geq 1 \text{ cm}$) in x

and y . For example, setting `SOLVER_MAIN_DIRECTIONS = 'z'` with $\Delta x = \Delta y = 1\text{ cm}$ and keeping $\Delta z = 0.01\text{ mm}$ yields a stability limit of $\Delta t < 5\text{ s}$, which is more than sufficient for most simulations.

The parameter `SOLVER_MAIN_DIRECTIONS` can take the values `'x'`, `'y'`, `'z'`.

6.3. Subdividing the Mesh into Multiple Regions and Creating Complex Geometries

A mesh can be subdivided into multiple regions, each with its own initial conditions and boundary conditions (see Section 8.0). This is done using objects called obstructions. Each obstruction represents a rectangular parallelepiped region within the mesh. The total number of obstructions is specified using the parameter `NOBST` in the `&GPYRO_GENERAL` namelist.

For each obstruction `IOBST` (from 1 to `NOBST`), its volume is defined by the coordinates `X1(IOBST)`, `X2(IOBST)`, `Y1(IOBST)`, `Y2(IOBST)`, `Z1(IOBST)`, and `Z2(IOBST)`. These values delimit the obstruction's occupied space within the mesh. In 1D simulations, only `Z1` and `Z2` need to be specified. In 2D simulations, `X1`, `X2`, `Z1`, and `Z2` must be specified.

When multiple meshes are defined, the optional parameter `IMESH(IOBST)` can be used to specify the index of the mesh to which the obstruction belongs.

Obstructions are essential for modeling multilayer materials, as each obstruction can represent a distinct layer with its own material properties. They are also useful in modeling experiments such as cone calorimeter tests, where insulating substrates or support blocks (e.g., underlying bricks) need to be represented as separate volumes.

If two obstructions overlap, the last one defined in the input file takes precedence and will occupy the overlapping volume.

If at least one obstruction is defined, the computational domain will be restricted to the regions occupied by the obstructions. Any mesh cells outside all defined obstructions will be ignored by the solver. This feature enables modeling of complex geometries within a single mesh. For example, an I-beam can be represented by one mesh containing three obstructions, each corresponding to a distinct block of the beam.

6.4. Defining Initial Conditions

The initial condition for each mesh `IMESH` is specified through the parameter `DEFAULT_IC(IMESH)`. This value must be an integer corresponding to the index of an initial condition type defined in the `&GPYRO_IC` namelist. An initial condition type determines, for example, the initial composition of both solid and gas phases, the initial temperature, and the initial pressure, as described in Section 7.0.

When the geometry contains multiple obstructions, each obstruction `IOBST` can be assigned its own initial condition by setting the parameter `ICNUM(IOBST)`. This parameter works in the same way as `DEFAULT_IC` but applies specifically to a given obstruction, allowing different parts of the domain to start with different states.

6.5. Defining Boundary Conditions

Boundary conditions for a mesh are set with `DEFAULT_SURF_IDX(IMESH, 1:6)`, where the second index corresponds to one of the six faces of the rectangular domain. Each face index refers to a boundary condition defined in the `&GPYRO_ALLBC` namelist, such as temperature, pressure, gas composition (see Section 8.0). The correspondence between face indices and their spatial orientation is shown below:

Table 2: Orientation index of the boundary faces.

Index	Orientation	Direction
1	West	-x
2	East	+x
3	South	-y
4	North	+y
5	Top	+z
6	Bottom	-z

For one-dimensional simulations, only faces 5 and 6 need to be specified; in two dimensions, faces 1, 2, 5, and 6 are required; in three dimensions, all six faces must be assigned a boundary type.

Obstructions can also have individual boundary conditions assigned through the parameter `SURF_IDX2D(IOBST, 1:6)`. This follows the same logic as for meshes. If two obstructions are in contact with a given face, no boundary condition needs to be set for that face, as it does not represent an external boundary.

6.6. Examples of Geometry Definitions

This section provides illustrative examples of mesh definitions using the `&GPYRO_GEOM` namelist in Gpyro. Each example demonstrates typical configurations for different simulation scenarios.

Example 1 : Simple 1D Block

This example defines a single 1D block of 5 cm thickness, refined in the z-direction with 50 cells. No obstructions are used; the computational domain corresponds to the entire mesh.

```
&GPYRO_GEOM
  NMESH = 1
  ZDIM(1) = 0.05
  NCELLZ(1) = 50
```

```

DEFAULT_IC (1) = 1
DEFAULT_SURF_IDX(1,5) = 1
DEFAULT_SURF_IDX(1,6) = 2/

```

Note that, since Gpyro V2, for 1D cases, XDIM, YDIM, NCELLX, and NCELLY default to 1 and do not need to be defined. This is not the case with older versions.

Example 2 : 3D Block with Multiple Layers

A 3D mesh is defined with three layers represented as obstructions. Each layer has its own initial condition index, typical for simulating multilayer materials.

```

&GPYRO_GEOM
  NMESH = 1
  NOBST = 3

  ! Mesh dimensions (m)
  XDIM(1) = 0.10
  YDIM(1) = 0.10
  ZDIM(1) = 0.04

  ! Number of cells
  NCELLX(1) = 20
  NCELLY(1) = 20
  NCELLZ(1) = 400

  ! Default initial condition for whole mesh
  DEFAULT_IC(1) = 1

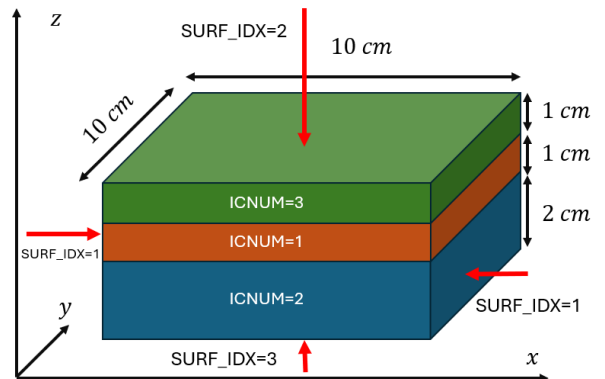
  ! Boundary conditions for six faces
  DEFAULT_SURF_IDX(1,1) = 1
  DEFAULT_SURF_IDX(1,2) = 1
  DEFAULT_SURF_IDX(1,3) = 1
  DEFAULT_SURF_IDX(1,4) = 1
  DEFAULT_SURF_IDX(1,5) = 2
  DEFAULT_SURF_IDX(1,6) = 3

  ! Obstructions representing layers
  ! Top layer (1 cm)
  X1(1) = 0.0 ; X2(1) = 0.10
  Y1(1) = 0.0 ; Y2(1) = 0.10
  Z1(1) = 0.03 ; Z2(1) = 0.04
  ICNUM(1) = 3

  ! Middle layer (1 cm)
  X1(2) = 0.0 ; X2(2) = 0.10
  Y1(2) = 0.0 ; Y2(2) = 0.10
  Z1(2) = 0.02 ; Z2(2) = 0.03
  ICNUM(2) = 1

  ! Bottom layer (1 cm)
  X1(3) = 0.0 ; X2(3) = 0.10
  Y1(3) = 0.0 ; Y2(3) = 0.10
  Z1(3) = 0.0 ; Z2(3) = 0.02

```



```
ICNUM(3) = 2/
```

Notes that in older versions of Gpyro require SURFIDX2D definitions to override default initial conditions, since Gpyro V2, these are only needed if modifications to the default are required.

Example 3 : I-Beam Geometry

This example defines an I-beam within a single mesh using three obstructions: one vertical web and two horizontal flanges.

```
&GPYRO_GEOM
  NMESH = 1
  NOBST = 3

  ! Mesh dimensions (m)
  XDIM(1) = 0.20
  YDIM(1) = 0.20
  ZDIM(1) = 0.20

  ! Number of cells
  NCELLX(1) = 100
  NCELLY(1) = 100
  NCELLZ(1) = 100

  ! Default initial condition
  DEFAULT_IC(1) = 1

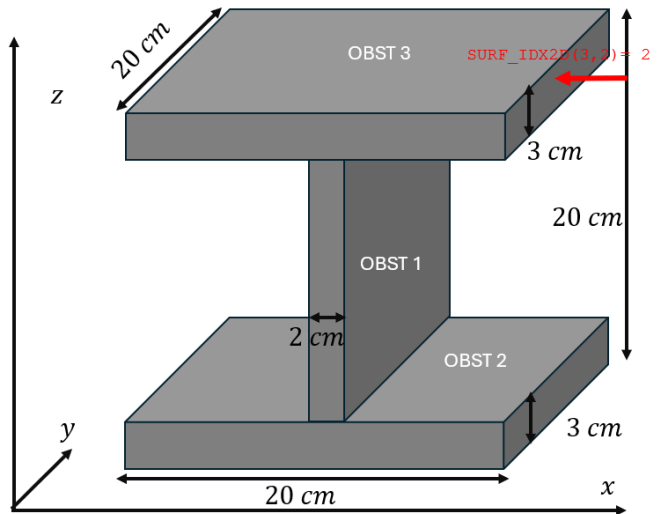
  ! Boundary conditions for all six faces
  DEFAULT_SURF_IDX(1,1) = 1
  DEFAULT_SURF_IDX(1,2) = 1
  DEFAULT_SURF_IDX(1,3) = 1
  DEFAULT_SURF_IDX(1,4) = 1
  DEFAULT_SURF_IDX(1,5) = 1
  DEFAULT_SURF_IDX(1,6) = 1

  ! Obstruction 1 – vertical web
  X1(1) = 0.09 ; X2(1) = 0.11
  Y1(1) = 0.00 ; Y2(1) = 0.20
  Z1(1) = 0.00 ; Z2(1) = 0.20
  ICNUM(1) = 1

  ! Obstruction 2 – bottom flange
  X1(2) = 0.00 ; X2(2) = 0.20
  Y1(2) = 0.00 ; Y2(2) = 0.20
  Z1(2) = 0.00 ; Z2(2) = 0.03
  ICNUM(2) = 1

  ! Obstruction 3 – top flange
  X1(3) = 0.00 ; X2(3) = 0.20
  Y1(3) = 0.00 ; Y2(3) = 0.20
  Z1(3) = 0.17 ; Z2(3) = 0.20
  ICNUM(3) = 1
  ! Different Boundary condition on this surface
  SURF_IDX2D(3,2) = 2
```

/



7.0 INITIAL CONDITIONS

The initial conditions are defined in the namelist group `&GPYRO_IC` (See Table 9). This group specifies the physical state assigned to a region at the beginning of the simulation (See Section 6.4). The `&GPYRO_IC` namelist provides a flexible way to initialize both simple homogeneous states and more complex material mixtures, including gas-phase composition when required by the model configuration. The total number of distinct initial condition sets is specified by the parameter `NIC`. Each set is identified by its index `IC` and may include several properties.

7.1. Condensed-phase temperature

`TMP_INITIAL(IC)` defines the initial temperature of the solid phase in kelvin. The default value is 300 K.

7.2. Gas-phase temperature

`TMPG_INITIAL(IC)` defines the initial temperature of the gas phase, also in kelvin. The default value is 300 K. This parameter is only relevant if the gas-phase energy conservation equation is solved (see Section 4.5).

7.3. Pressure

`P_INITIAL(IC)` specifies the initial pressure in pascals. The default value is 101 300 Pa. This parameter is only considered when pressure equations are solved (see Section 4.3).

7.4. Condensed species mass fractions

For a condensed species of index `ISPEC` defined in the `&GPYRO_SPROPS` namelist (see Section 9.0), its initial mass fraction in kg/kg is defined through `YI0(IC, ISPEC)`. By default, all fractions are zero, and the sum over all indices from 1 to `NSPEC` must equal one. This feature allows a material to be represented either as a pure component or as a mixture of species.

7.5. Gas species mass fractions

Similarly, the initial gas composition can be defined using `YJ0(IC, IGSPEC)`, which specifies the mass fraction of gaseous species `IGSPEC` (see Section 10.1). This represents the mass of a given gas species relative to all gas species, not the mass of gas relative to the total mass (solid + gas). The default is zero, and the sum over all species indices from 1 to `NGSPEC` must also equal one. This option is only relevant if gas advection–diffusion is activated (see Section 4.4).

7.6. Example

The following example defines two distinct initial condition sets.

```
&GPYRO_IC NIC=2
      TMP_INITIAL(1) = 273.15
```

```
YI0 (1,1)=0.7  
YI0 (1,3)=0.3  
  
TMP_INITIAL(2)= 273.15  
YI0 (2,3)=1
```

/

Here, two sets are introduced (NIC=2). The first set assigns a condensed-phase temperature of 273.15 K, with the material represented as a mixture of 70% species 1 and 30% species 3. The second set also has an initial temperature of 273.15 K, but the material is composed entirely of species 3.

8.0 BOUNDARY CONDITIONS

Boundary conditions define the interaction between the computational domain and its external environment. They are not required in 0D calculations, since the governing equations reduce to ordinary differential equations that only depend on the initial conditions. For higher-dimensional simulations (1D, 2D, 3D), boundary conditions must be specified for each exposed surface of the mesh (See Section 6.5). Boundary conditions are defined in the namelist group `&GPYRO_ALLBC` (see Table 10).

8.1. Time-dependent boundary condition definition

The boundary conditions are defined as a sequence of temporal blocks. The total number of blocks is set by `NSURF_IDX`, and each block is indexed by `IBC = 1...NSURF_IDX`. For every block, the keyword `SURF_IDX (IBC)` assigns a boundary surface index, which links several temporal blocks into a single boundary condition. Each block is also associated with a specific time, defined by `T (IBC)` in seconds. At any instant lying between the times of two consecutive blocks, the applied boundary condition is obtained by linear interpolation of the values defined in those blocks. For times larger than the last time block associated with a given boundary surface index, the boundary condition remains constant and equal to the values prescribed in the final block. This formulation provides flexibility to represent evolving environments, such as ignition, where an additional heat flux can be applied to account for flame exposure.

As discussed in the geometry definition (see Section 6.5), the parameters `DEFAULT_SURF_IDX` and `SURF_IDX2D` must reference one of boundary surface index defined in `&GPYRO_BC`. This mechanism ensures that each geometric surface is consistently associated with the correct boundary condition, which may itself evolve according to its temporal blocks.

8.2. Thermal boundary conditions (condensed phase)

At the solid surface, two types of thermal boundary conditions can be imposed.

8.2.1. Fixed temperature

A fixed surface temperature can be prescribed through the parameter `TFIXED (IBC)`. If the value is negative, no fixed temperature is applied, which corresponds to the default setting.

8.2.2. Imposed heat flux

Alternatively, a boundary condition can be defined by imposing a heat flux on the surface. The total heat flux is expressed as the sum of three contributions: convection, external radiation, and re-radiation.

Convection

The convective heat flux at the boundary is written as $h_c(T_{inf} - T_{surf})$, where h_c is the convective heat transfer coefficient (in $\text{W}\cdot\text{m}^{-1}\cdot\text{K}^{-1}$) specified by the parameter `HC (IBC)`, T_{inf} is the external gas temperature defined by `TINF (IBC)`, and T_{surf} is the surface temperature.

Gpyro also provides an optional correction to account for the effect of pyrolysis gas blowing on convective heat transfer. In this formulation, the effective convective heat transfer coefficient is modified using a Couette-flow approximation [9]:

$$h_c = \frac{\dot{m}_{gas}'' c_{pg}}{\exp\left(\frac{\dot{m}_{gas}'' c_{pg}}{h_{c,0}}\right) - 1} \quad (21)$$

where $\dot{m}_{gas}''$ is the outward gas mass flux, c_{pg} is the gas heat capacity, and $h_{c,0}$ is the convective heat transfer coefficient in the absence of blowing. This formulation is activated by setting `BLOWING = .TRUE.` in the `&GPYRO_GENERAL` namelist group. When this option is enabled, the parameter `HC (IBC)` is interpreted as the reference coefficient $h_{c,0}$.

External radiation

An external radiative flux can be prescribed via `QE (IBC)` (in $\text{W}\cdot\text{m}^{-2}$). Only a fraction of this flux is absorbed, depending on the surface emissivity (see Section 9.2.4). The absorbed energy can either be deposited directly at the surface or, for more realistic semi-transparent materials, distributed within a penetration depth inside the solid (see Section 9.2.4). This option can, for example, reproduce the radiative heat flux imposed in cone calorimeter experiments.

Re-radiation

The surface also exchanges thermal radiation with its environment. This is represented through the Stefan–Boltzmann law written as $q_{rerad} = \epsilon\sigma(T_{inf}^4 - T_{surf}^4)$, where ϵ is the surface emissivity and σ is the Stefan–Boltzmann constant. This contribution is enabled when the parameter `RERADIATION (IBC) = .TRUE.`. Since re-radiation is a physical process that always occurs, the default value of this parameter is `.TRUE.`

8.3. Thermal boundary conditions (gas phase)

When the gas-phase energy conservation equation is solved (See section 4.5), similar types of thermal boundary conditions apply. In this case, the corresponding parameters are `QEG (IBC)` for external flux, `HCG (IBC)` for convection coefficient, `TINFG (IBC)` for external gas temperature, and `TFIXEDG (IBC)` for imposed fixed temperature. These parameters play roles analogous to those used in the condensed phase but are applied to the gas temperature field.

8.4. Gas transport boundary conditions

8.4.1. Instantaneous release boundary condition

In the simplest formulation, the gas generated inside the material is assumed to be instantaneously released at the surface. In this case, it is optionally possible to impose a gas inflow at the bottom surface ($-z$ direction) using the parameter $\text{MDOTPP}(\text{IBC})$, which specifies the gas flow rate in $\text{g}\cdot\text{m}^{-2}\cdot\text{s}^{-1}$. This can represent, for example, gas injection in fixed- or fluidized-bed configurations, or controlled gas supply in experimental setups. The default value of MDOTPP is zero, and in this case only positive values are allowed. This additional inflow can only be imposed at the bottom surface.

The gas enters the domain at a specified temperature, which can be imposed through the parameter $\text{TMPG}(\text{IBC})$. When this option is used, the temperature difference between the injected gas and the solid is automatically taken into account by the thermal solver.

8.4.2. Pressure-driven gas flow boundary condition

When the pressure solver is activated (see Section 4.3), boundary conditions can be prescribed in two different ways:

Imposed pressure

The pressure at a boundary can be prescribed directly using $\text{PRES}(\text{IBC})$ (in Pa). In this case, the mass flux across the boundary is computed consistently with Darcy's law, as a function of the pressure gradient and the material permeability.

Imposed mass flux

As in the instantaneous-release case, a gas flux is imposed through $\text{MDOTPP}(\text{IBC})$ (in $\text{g}\cdot\text{m}^{-2}\cdot\text{s}^{-1}$) and by setting $\text{PRES}(\text{IBC})$ to an arbitrary negative value. This flux can be specified across any surface. Positive values correspond to inflow, whereas negative values represent outflow. Setting $\text{MDOTPP}(\text{IBC})=0$ corresponds to an impermeable boundary condition.

8.5. Gas species transport boundary conditions

When the optional species advection–diffusion solver is activated (see Section 4.4), the gas-phase species boundary condition is determined by whether an external mass flux is prescribed.

Inlet with prescribed mass flux

If a non-zero input mass flux is imposed at a boundary ($\text{MDOTPP}(\text{IBC}) \neq 0$), the solver uses the external gas-phase composition specified by $\text{YJINF}(\text{IBC}, J)$ to define the incoming species fractions for each gaseous species J .

$$Y_{j,surf} = Y_{j,inf} \quad (22)$$

Boundary without prescribed mass flux

If no mass flux is prescribed ($MDOTPP(IBC) = 0$), the boundary condition follows a convective transfer relation :

$$-\bar{\psi}\rho_g D \frac{dY_j}{dz} = h_m(Y_{j,inf} - Y_{j,surf}) \quad (23)$$

where $Y_{j,inf}$ is the mass fraction of species j in the external gas phase, it is again taken from the user-specified $YJINF(IBC, J)$ values, and $Y_{j,surf}$ is the species mass fraction computed at the material surface. h_m is the mass transfer coefficient ($\text{kg}\cdot\text{m}^{-2}\cdot\text{s}^{-1}$) is provided through the parameter $HM(IBC)$.

By default, all $YJINF$ values are set to zero, and the user must ensure that their sum over the $NGSPEC$ species equals to one.

Practical guidance

At an Outflow surfaces, in most practical cases, advection dominates over diffusion (high Peclet number). For such surfaces, a free-outlet formulation is preferred to allow the gas to exit freely:

$$\frac{dY_j}{dz} = 0 \quad (24)$$

To implement this, set $HM(IBC) = 0$.

In Diffusion-dominated cases (e.g., very slow smoldering, gentle airflow), a starting estimate for h_m can be obtained using the heat/mass transfer analogy:

$$h_m = \frac{h_c}{c_{pg}} \quad (25)$$

8.6. Example

```
&GPYRO_ALLBC
  NSURF_IDX=7

  SURF_IDX(1) = 1
  HC(1)        = 0
  QE(1)        = 0
  RERADIATION(IBC) = .FALSE.

  SURF_IDX(2) = 2
```

```

T (2)          = 0
HC (2)         = 10
TINF (2)       = 300
QE (2)         = 50

SURF_IDX (3) = 2
T (3)         = 49
HC (3)        = 10
TINF (3)      = 300
QE (3)        = 50000

SURF_IDX (4) = 2
T (4)         = 50
HC (4)        = 10
TINF (4)      = 300
QE (4)        = 65000

SURF_IDX (5) = 2
T (5)         = 449
HC (5)        = 10
TINF (5)      = 300
QE (5)        = 65000

SURF_IDX (6) = 2
T (6)         = 450
HC (6)        = 10
TINF (6)      = 300
QE (6)        = 50000

SURF_IDX (7) = 3
TFIXED (7)    = 800

```

/

In this example, SURF_IDX = 1 is defined with a single temporal block (IBC =1). The condition corresponds to an adiabatic surface: convection is disabled with $HC(1) = 0$, radiation is set to zero, and re-radiation is deactivated.

SURF_IDX = 2 is defined with five temporal blocks (IBC from 2 to 6). This boundary condition reproduces the heat flux imposed in a cone calorimeter test. Initially, a radiative flux of $50 \text{ kW}\cdot\text{m}^{-2}$ is applied. At 50 s, ignition is assumed to occur, and the flame adds an additional $15 \text{ kW}\cdot\text{m}^{-2}$, leading to a total radiative flux of $65 \text{ kW}\cdot\text{m}^{-2}$. At 450 s, extinction occurs, and the flux drops back to $50 \text{ kW}\cdot\text{m}^{-2}$. Convection is prescribed with $h_c = 10 \text{ W}\cdot\text{m}^{-2}\cdot\text{K}^{-1}$ and $T_{inf} = 300 \text{ K}$, while re-radiation is activated by default.

SURF_IDX = 3 (IBC=7) corresponds to a fixed-temperature condition, where the surface is maintained at 800 K.

9.0 CONDENSED-PHASE THERMOPHYSICAL PROPERTIES

9.1. Material Definition

Gpyro allows the user to define multiple condensed-phase materials, either solid or liquid. All materials initially present in the sample, as well as those generated during thermal degradation, must be declared. Material properties are specified in the &GPYRO_SPROPS namelist (See Table 11).

The total number of condensed-phase species is set using the parameter NSSPEC. Each species is assigned a unique name via the NAME (IDM) parameter, where IDM is the index of the material ranges from 1 to NSSPEC. Names should be concise, without spaces (underscores are allowed), e.g., wood, char, ash, glass_fibers.

9.2. Thermal properties

Each condensed-phase species, identified by its index IDM, must be assigned a set of thermophysical properties that govern heat transfer and thermal degradation, that include thermal conductivity, specific heat capacity, density, and radiative parameters.

9.2.1. Thermal conductivity

Thermal conductivity is defined independently in each spatial direction using the parameters KX (IDM), KY (IDM), and KZ (IDM), expressed in $\text{W.m}^{-1}.\text{K}^{-1}$. For one-dimensional simulations, only KZ is required. In two-dimensional cases, KX and KZ must be specified. Each conductivity component may vary with temperature according to a power-law expression:

$$k(T) = k_0 \left(\frac{T}{T_{ref}} \right)^n \quad (26)$$

where T_{ref} is a reference temperature defined in the &GPYRO_GENERAL namelist via the parameter TREF, and n is the exponent specified via NKX (IDM), NKY (IDM) or NKZ (IDM), depending on the direction.

In porous materials, radiative heat transfer within the solid phase may be optionally modeled by enhancing the effective thermal conductivity. This is achieved by adding a radiative term to the standard conductivity expression using the Rosseland approximation :

$$k_{eff} = k(T) + \gamma \sigma T^3 \quad (27)$$

Here, γ (unit in m) is an optional user-defined material-specific coefficient specified via the parameter GAMMA (IDM), and σ is the Stefan–Boltzmann constant.

9.2.2. Specific Heat Capacity

The specific heat capacity is defined using the parameter `C0 (IDM)`, in units ($\text{J} \cdot \text{kg}^{-1} \cdot \text{K}^{-1}$). As with conductivity, it may vary with temperature according to a power-law model. The exponent is specified via the parameter `NC (IDM)`.

To account for phase change (melting), Gpyro introduces an apparent heat capacity model (see the technical user guide Equation 9c). The melting behavior is controlled by three parameters, `TMELT (IDM)` the melting temperature in (K), `DHMELT (IDM)` the latent heat of melting in ($\text{J} \cdot \text{kg}^{-1}$), and `SIGMA2MELT (IDM)`, the variance of the Gaussian distribution in (K^2) that defines the width of the melting peak. If melting is not to be modeled, these parameters should not be used.

9.2.3. Bulk density

The bulk density of the material is specified via the parameter `R0 (IDM)` in (kg/m^3). It refers to the mass of the material per unit of total volume, including both the solid matrix and any internal voids or pores. In porous materials, this value represents the apparent density as seen at the macroscopic scale and therefore includes the contribution of porosity. It differs from the intrinsic or skeletal density, which accounts only for the mass and volume of the solid phase. As with other properties, temperature dependence may be introduced using a polynomial law, with the exponent defined by `NR (IDM)`.

Note that bulk density is coupled to the local deformation and stoichiometry of the reactions involving the species (See Section 11.2).

9.2.4. Radiative Properties

The condensed-phase species must also be assigned radiative properties that govern the emission and absorption of thermal radiation at the material surface. These include `EMIS (IDM)`, the surface emissivity (dimensionless), and `KAPPA (IDM)`, the absorption coefficient in (m^{-1}). The absorption coefficient defines the typical penetration depth of external radiation into the material. To model a surface that absorbs all incident radiation at the surface only, `KAPPA` should be set to a large value (e.g., $9.0\text{E}9$).

9.2.5. Example

This example illustrates the definition of two condensed-phase species: a polymeric foam and its degradation product, char. The thermal properties of each material are specified in the `GPYRO_SPROPS` namelist using indexed parameters.

The first material, `foam`, is defined with a constant thermal conductivity of $0.15 \text{ W} \cdot \text{m}^{-1} \cdot \text{K}^{-1}$. The bulk density is set to $40 \text{ kg} \cdot \text{m}^{-3}$, representing the apparent density including porosity. The specific heat capacity is temperature-dependent and follows a power-law relation $c_p(T) = 1200(T/$

$T_{ref})^{1.2} \text{ J}\cdot\text{kg}^{-1}\cdot\text{K}^{-1}$. The surface emissivity is set to 0.9, and the absorption coefficient of 2700 m^{-1} defines the penetration depth of external radiation into the material.

The second material, `char`, is defined with a base thermal conductivity of $0.1 \text{ W}\cdot\text{m}^{-1}\cdot\text{K}^{-1}$ and includes an additional radiative contribution modeled via the Rosseland approximation. The effective conductivity is given by $k_{eff} = 0.1 + 0.01\sigma T^3 \text{ W}\cdot\text{m}^{-1}\cdot\text{K}^{-1}$. The bulk density is $20 \text{ kg}\cdot\text{m}^{-3}$, and the specific heat $1200 \text{ J}\cdot\text{kg}^{-1}\cdot\text{K}^{-1}$. The emissivity is 0.85. To ensure that radiation is absorbed only at the surface, the absorption coefficient is set to a large value: $\text{KAPPA}(2) = 9\text{D}9$.

```
&GPYRO_SPROPS  NSSPEC = 2,

      NAME(1)    = 'foam',
      KOZ(1)     = 0.15,
      RO(1)      = 40,
      CO(1)      = 1200,
      NC(1)      = 1.2,
      EMIS(1)    = 0.9,
      KAPPA(1)   = 2700,

      NAME(2)    = 'char',
      KOZ(2)     = 0.1,
      GAMMA(2)   = 0.01
      RO(2)      = 20,
      CO(2)      = 1200,
      EMIS(2)    = 0.85,
      KAPPA(2)   = 9D9/
```

9.3. Transport Properties in Porous Media

The following parameters describe the ability of a condensed-phase material to resist gas transport. These properties are relevant only when modeling gas flow within porous solids. They are active only if the gas-phase transport equation is solved, i.e., when `SOLVE_PRESSURE = .TRUE.` in the `GPYRO_GENERAL` namelist. *If gas transport is not considered, these parameters may be omitted.*

9.3.1. Solid density

The intrinsic solid density, denoted ρ_s , is specified via the keyword `RS0 (IDM)` in units of $(\text{kg}\cdot\text{m}^{-3})$. This value represents the true density of the solid matrix, excluding porosity, and is distinct from the bulk density ρ , which includes the volume of pores. The porosity of each species is computed as $\psi_i = 1 - \rho_i/\rho_{s,i}$. The mixture-averaged porosity is then calculated as $\bar{\psi} = \sum X_i \psi_i$.

9.3.2. Permeability

The intrinsic permeability of the porous material i ($K_i = (K_{ix}, K_{iy}, K_{iz})$) is defined using the parameters `PERMZ (IDM)`, `PERMX (IDM)`, and `PERMY (IDM)` for the z , x , and y directions, respectively. It is expressed in m^2 and determines the resistance of the porous matrix to gas flow. the mixture-averaged permeability is computed as $\bar{K} = \sum X_i K_i$.

An alternative option is activated by setting `KOZENY_CARMAN=.TRUE.` in the `GPYRO_GENERAL` namelist (default: `.FALSE.`). In this case, permeability evolves with porosity according to the Kozeny–Carman model that link porosity to permeability [ref]:

$$\bar{K} = \frac{\bar{\psi}^3}{(1 - \bar{\psi})^2} \sum X_i K_i \quad (28)$$

9.3.3. Pore diameter

The mean pore diameter (d_p in m), is specified via the keyword `PORE_DIAMETER (IDM)`. This parameter is used in the computation of the volumetric heat transfer coefficient between the gas and solid phases. The coefficient is derived from a Nusselt number correlation that depends on the pore diameter and is discussed in the technical reference guide.

10.0 GAS-PHASE THERMOPHYSICAL PROPERTIES

Parameters controlling gas-phase thermophysical properties are specified via the &GPYRO_GPROPS Namelist group. A summary of all parameters is presented in

Table 12. This namelist is used to define the gaseous species produced or reacting during pyrolysis, as well as non-reacting species such as those in the ambient atmosphere (e.g., nitrogen, oxygen).

10.1. Gaseous species definition

The total number of gaseous species is set with the keyword `NGSPEC`, which must be at least equal to 1, even if the gaseous species conservation equation is not explicitly solved.

Each gaseous species is identified by an index `IGAS` (from 1 to `NGSPEC`) and named via the parameter `NAME(IGAS)`. This parameter is a character string that must be unique for each species (no spaces are allowed). Examples include 'oxygen', 'nitrogen', or 'pyrolysate1'.

10.2. Special oxidative species

In the case of oxidative reactions, it is necessary to specify which gaseous species acts as the oxidizer, most commonly oxygen. The keyword `IO2` assigns the index of the oxidizer species. This parameter is required whenever oxidative reactions are included (See Section 11.4.3).

10.3. Gas heat capacity

The gas-phase specific heat capacity ($\text{J}\cdot\text{kg}^{-1}\cdot\text{K}^{-1}$) is prescribed via the keyword `CPG`. For simplicity, all gaseous species are assumed to share the same heat capacity, independent of temperature. This parameter is required only when gas/solid heat exchange is modeled or when the gas-phase energy conservation equation is solved. As an example, the heat capacity of air is approximately $1000 \text{ J}\cdot\text{kg}^{-1}\cdot\text{K}^{-1}$, which is the default value.

10.4. Gas molecular weight

The molecular weight of each species ($\text{g}\cdot\text{mol}^{-1}$) is specified via the keyword `M(IGAS)`. This property links pressure, density, and temperature through the ideal gas law ($\rho_g = RMPT_g$) where M is the molar mass, R is the universal gas constant, and T_g is the temperature. For reference, the molecular weight of air is $28.96 \text{ g}\cdot\text{mol}^{-1}$, which is used as the default value.

10.5. Diffusive gas properties

All gas species are assigned the same mass diffusivity D (in $\text{m}^2\cdot\text{s}^{-1}$), which is taken as the diffusivity of the oxidative species in a background gas. The diffusivity is calculated using the Chapman–Enskog theory [ref]. It depends on pressure P and Temperature T and is given by:

$$\nu = D = 0.018829 \frac{\sqrt{T^3 \left(\frac{1}{M_A} + \frac{1}{M_B} \right)}}{P \sigma_{AB}^2 \Omega_{D,AB}} \quad (29)$$

where M_A and M_B are the molar masses of the oxidative species and the background species (See Section 10.4). σ_{AB} and $\Omega_{D,AB}$ are parameters derived from the properties of the oxidative and background species. They are defined as:

$$\sigma_{AB} = \frac{1}{2}(\sigma_A + \sigma_B) \quad (30)$$

$$\Omega_{D,AB} = \alpha_1 (T^*)^{\alpha_2} + \alpha_3 \exp(\alpha_4 T^*) + \alpha_5 \exp(\alpha_6 T^*) \quad (31)$$

$$T^* = \frac{T}{\epsilon/k_{AB}} \quad ; \quad \epsilon/k_{AB} = \sqrt{\epsilon/k_A \epsilon/k_B}$$

with constants:

$$\begin{aligned} \alpha_1 &= 1.080794 \quad ; \quad \alpha_2 = -0.16033 \quad ; \quad \alpha_3 = 0.605009 \\ \alpha_4 &= -0.88524 \quad ; \quad \alpha_5 = 2.115672 \quad ; \quad \alpha_6 = -2.98308 \end{aligned}$$

Here, σ and ϵ/k are the intrinsic Lennard–Jones parameters of the oxidative and background species.

In the `&GPYRO_GPROPS` Namelist, the index of the background species is specified by `IBG`, and the index of the oxidative species by `IO2` (see section 10.2 The Lennard–Jones parameters `SIGMA (IGAS)` (σ specified in Å) and `EPSOK (IGAS)` (ϵ/k specified in K) are required only for these two species. Reference values for common gases are provided in Appendix C. Gas properties and are taken from (Bird et al. [ref]).

10.6. Gas viscosity

In Gpyro, the gas kinematic viscosity ν (in $\text{m}^2.\text{s}^{-1}$) is assumed equal to the gas mass diffusivity D ($Sc = \nu/D = 1$). This approximation is commonly used for gases. Consequently, the gas viscosity required for Darcy’s law is taken as the diffusivity values described in Section 10.5).

10.7. Example

The following is a sample `&GPYRO_GPROPS` line, consistent with the `&GPYRO_GYIELDS` example line given in the next sections.

```
&GPYRO_GPROPS NGSPEC      = 4
  IBG              = 3
  IO2              = 2
  CPG              = 1000.

  NAME(1) = 'pyrolysate', M(1) = 100.
  NAME(2) = 'oxygen'      , M(2) = 32. , SIGMA(2)=3.433, EPSOK(2)=113
  NAME(3) = 'nitrogen'    , M(3) = 28.2, SIGMA(3)=3.681, EPSOK(3)=91.5
  NAME(4) = 'products'    , M(4) = 44. /
```

11.0 CONDENSED-PHASE REACTIONS

Condensed-phase reactions describe the transformation of condensed species (solid or liquid) into other condensed products and gaseous species. Pure gas-phase reactions can also be modeled but are presented separately (see Section 12.0). This section focuses on condensed-phase reactions.

Gpyro allows the definition of multiple condensed phase reactions, which may occur in parallel, sequentially, or in mixed schemes. These reactions can represent various processes, including oxidative or catalytic pathways, phase changes, or purely thermal decomposition. Several kinetic models are available, offering flexibility in describing different reaction mechanisms.

Input parameters are provided through two namelist groups:

- `&GPYRO_RXNS`, which specifies the number of reactions, the condensed species involved (reactant and product), kinetic parameters, oxygen dependence, deformation factors, and reaction enthalpies (See Table 13).
- `&GPYRO_GYIELDS`, which defines the gaseous species consumed or produced by each reaction, and their relative stoichiometric proportions (species-yield matrix) (See Table 14).

The formulation and parameter definitions follow the formalism described in the Technical Reference (Chapter 3) [ref].

11.1. Declaring Condensed-Phase Reactions

11.1.1. Declaring a reaction

The total number of condensed-phase reactions is defined in the namelist `&GPYRO_RXNS` through the keyword `NRXNS`. Each reaction is identified by its index `IRXNS` from 1 to `NRXNS`.

In Gpyro , the general formulation of a condensed-phase reaction k is written as:



Where A is the only condensed reactant, B is the only condensed product, G represents all gaseous species consumed or generated by the reaction, and θ_k is the stoichiometric coefficient of the condensed product (See Section 11.2)

The condensed reactant (A) name is specified with `CFROM(IRXNS)` and that of condensed product (B) with `CTO(IRXNS)`. Both names must exactly match species names declared in `&GPYRO_SPROPS` and are case-sensitive. Associated gaseous products or reactants, as well as their stoichiometry, are defined separately in `&GPYRO_GYIELDS` (See section 12.0).

11.1.2. Special case: non-charring reactions

In the default formulation, every reaction must produce a condensed product. However, in some cases the condensed phase volatilizes completely (non-charring reactions), and no solid residue is left. This type of decomposition can be represented by assigning the keyword `CTO (IRXNS) = 'gases'`. When this option is used, the entire mass of the condensed reactant is converted into gaseous species, whose distribution is then specified in `&GPYRO_GYIELDS`.

11.1.3. Special case: multiple condensed products

Since Gpyro restricts each reaction to a single condensed product, reactions that conceptually generate several condensed products must be decomposed into multiple parallel reactions. Each parallel reaction represents the formation of one of the products, while sharing the same kinetic parameters (see Section 11.4) and enthalpy of reaction (see Section 11.5) as the original process. The stoichiometric coefficients must then be adjusted individually to reproduce the correct mass balance for each product (See Section 11.2).

11.2. Condensed-Phase Stoichiometry and Deformation

11.2.1. Definition of the Stoichiometric Coefficient

As seen in Equation (32) separates the fraction of the condensed reactant mass that remains as condensed product from the fraction converted into gas ($1 - \theta_k$). The detailed stoichiometry of gaseous species is discussed in Section 11.4 here, the focus is placed on the condensed phase.

In Gpyro, θ_k is not imposed directly. Instead, it is determined from the densities of the reactant and product, together with a deformation factor χ_k , according to:

$$\theta_k = 1 + \left(\frac{\rho_B}{\rho_A} - 1 \right) \chi_k \quad (33)$$

where ρ_A and ρ_B are the densities of the condensed reactant and product (see Section 9.2.3). The parameter χ_k , which is imposed via the keyword `CHI (IRXNS)`, gives the user explicit control over deformation, and together with the densities, determines the value of θ_k .

11.2.2. Deformation and Volume Change

The volumetric deformation induced by a reaction can be quantified as the ratio between the final volume after reaction (V_f) and the initial volume before reaction (V_{ini}) and it is noted $\mathfrak{D}_k$. This ratio is expressed in terms of densities and stoichiometry as:

$$\mathfrak{D}_k = \frac{V_f}{V_{ini}} = \frac{\rho_A m_B}{\rho_B m_A} = \frac{\rho_A \theta_k}{\rho_B} \quad (34)$$

Thus, given the density of the reactant (ρ_A) and the stoichiometric coefficient (θ_k), the desired deformation can be reproduced by selecting an appropriate value of ρ_B to respect Equation (33). The corresponding value of χ_k is then deduced from Equation (34) **Erreur ! Source du renvoi introuvable.** to ensure consistency between stoichiometry and deformation.

Some values of the deformation factor χ_k correspond to characteristic physical situations:

- When $\chi_k = 0$, then $\theta_k = 1$ (Equation (33)), meaning that the reaction is **purely solid–solid**, and the resulting deformation is governed solely by the density ratio of reactant and product.
- When $\chi_k = 1$, then $\theta_k = \rho_B/\rho_A$. In this case, the reaction corresponds to a typical charring process with **no net volume change**, as deduced from Equation (34).
- If $\chi_k < 1$ and $\rho_B < \rho_A$, the material **shrinks**, meaning its volume decreases. The smaller χ_k is, the more pronounced the shrinkage.
- If $\chi_k > 1$ and $\rho_B < \rho_A$, the material **swells**, leading to an increase in volume. The larger χ_k is, the stronger the swelling effect.

11.3. Gaseous stoichiometry: the species-yield matrix GYIELDS

The gaseous stoichiometry for each condensed-phase reaction is specified in the &GPYRO_GYIELDS namelist through the parameter GYIELDS (IGAS, IRXNS). These entries form the species yield matrix $y_{s,j,k}$ described in the Technical Reference [ref].

As shown in Equation 6, for a given reaction k , the total fraction of the reactant mass converted into gases is $(1 - \theta_k)$. The contribution of a particular gaseous species j is:

$$\theta_{k,j} = y_{s,j,k}(1 - \theta_k) \quad (35)$$

Here, $Y_{s,j,k}$ corresponds to GYIELDS (IGAS, IRXNS). A positive value indicates that the species is produced, a negative value means it is consumed (acting as a reactant), and a value of zero means the species does not participate in the reaction, which is the default. For each reaction IRXNS, the sum of all GYIELDS (:, IRXNS) entries must equal 1 to ensure mass conservation.

By defining this matrix, it becomes possible to fully specify which gaseous species participate in each reaction, both as reactants and as products, as well as their stoichiometric proportions.

11.4. Kinetics of Condensed-Phase Reactions

11.4.1. General formulation

The rate of a condensed-phase reaction k , denoted r_k (in $\text{kg.m}^{-3}.\text{s}^{-1}$), represents the destruction rate of the condensed reactant A . In Gpyro, this rate is expressed as:

$$r_k = g_k(\alpha_A)k_k(T)g_k(Y_{O_2}) \quad (36)$$

where:

- $g_k(\alpha_A)$ is the kinetic model function, which depends on the conversion degree of the species A noted as α_A .
- $k_k(T)$ is the temperature-dependent rate constant, given by the Arrhenius law.
- $g_k(Y_{O_2})$ accounts for the effect of local oxidative concentration (if applicable).

The most common case is thermal decomposition without oxidation, where $g_k(Y_{O_2}) = 1$.

Note: *The kinetic description and associated parameters in Gpyro are not necessarily equivalent to those found in other literature sources. It is strongly recommended to validate kinetic data against Thermogravimetric Analysis (TGA) experiments to ensure consistency with the intended behavior. Given the complexity of these parameters, performing inverse modeling on TGA data can be an effective strategy to obtain values that match experimental observations (see Chapter 14.0).*

11.4.2. Temperature Dependence (Arrhenius Law)

The influence of temperature on the reaction rate constant is described by the Arrhenius expression:

$$k_k(T) = Z_k \exp\left(-\frac{E_k}{RT}\right) \quad (37)$$

In this formulation, Z_k represents the pre-exponential factor in s^{-1} , which is specified for each reaction through the parameter Z (IRXNS). The term E_k corresponds to the activation energy in $kJ.mol^{-1}$ and is defined by E (IRXNS). The constant R is the universal gas constant, and T is the local temperature in Kelvin.

11.4.3. Kinetic models

The kinetic term in the reaction rate expression accounts for how the availability of the condensed reactant and its conversion state influence the reaction progress. In Gpyro, this term is written as:

$$g_k(\alpha_A) = f_k(\alpha_A) \alpha_A^{m_k} \frac{\bar{\rho} Y_A}{(1 - \alpha_A)} \quad (38)$$

Available models

Here, $f_k(\alpha_A)$ represents the kinetic model function, it can take different forms depending on the kinetic model selected. This choice is controlled by the parameter $IKINETICMODEL$ (IRXNS), which refers to an index in the kinetic model presented in Table 3. 3. The default model is the n -th order model ($IKINETICMODEL$ (IRXNS) = 0), which is the most commonly used one.

For models that require an order parameter (n_k), the user must specify $ORDER$ (IRXNS). For catalytic reactions ($IKINETICMODEL = 9$), additional parameter $KCAT$ (IRXNS) (catalytic constant) and $ICAT$ (IRXNS) (index of the catalytic species) are required.

Additionally in Equation (38), the parameter m_k specified by M (IRXNS) introduces an additional exponent on α_A that can be used to adjust the shape of the reaction rate curve. The default value is zero

Table 3: Available kinetic models in Gpyro.

IKINETICMODEL	Description	$f_k(\alpha_A)$
0	n^{th} order	$(1 - \alpha_A)^{n_k}$
1	Nucleation and nucleus growing	$\frac{1}{n_k} (1 - \alpha_A) [-\ln(1 - \alpha_A)]^{n_k}$
2	Phase boundary reaction	$(1 - \alpha_A)^{n_k}$
3	Diffusion – plane symmetry	$\frac{1}{2\alpha_k}$
4	Diffusion – cylindrical symmetry	$\frac{1}{-\ln(1 - \alpha_A)}$
5	Diffusion – spherical symmetry	$\frac{3}{2} \frac{1}{(1 - \alpha_A)^{-1/3} - 1}$
6	Diffusion – Jander’s type	$-\frac{3}{2} \frac{(1 - \alpha_A)^{-1/3}}{\alpha_A}$
7	Potential law	$\frac{1}{n_k} \alpha_A^{1-n_k}$
8	Reaction order	$\frac{(1 - \alpha_A)^{1-n_k}}{n_k}$
9	Catalytic	$(1 - \alpha_A)^{n_k} (1 + K_{cat,k} \alpha_{icat})$

The Conversion Rate

In Gpyro, the conversion degree of a condensed species, denoted as α_A , represents the fraction of the initial material that has reacted. It varies from 0 (completely unreacted) to 1 (fully consumed). Two formulations are available to compute this quantity (See Equation (39)).

The default and most general formulation is activated when `CONVENTIONAL_RXN_ORDER = .FALSE.` in the `&GPYRO_GENERAL` namelist. This approach ensures that α_A remains within the $[0, 1]$ range for all species, including those produced as intermediate products. It provides a physically consistent definition of conversion degree, especially in complex reaction networks.

In contrast, the alternative formulation, which is closer to that used in other pyrolysis codes such as FDS, is selected by setting `CONVENTIONAL_RXN_ORDER = .TRUE.` While this version also keeps α_A between 0 and 1, it does not necessarily span the full range from 0 to 1. As a result, it may underestimate the extent of conversion, particularly for intermediate species, and does not fully capture the intended definition of the conversion degree. Unless you have a specific reason to use the alternative formulation, it is strongly recommended to keep `CONVENTIONAL_RXN_ORDER = .FALSE.`

$$(1 - \alpha_A) = \begin{cases} \frac{m_a(t)}{m_0(t=0)} & \text{if } \text{CONVENTIONAL_RXN_ORDER} = \text{TRUE.} \\ \frac{m_a(t)}{m_a(t=0) + \int_0^t \dot{\omega}_{f,a}'''' \Delta V d\theta} & \text{if } \text{CONVENTIONAL_RXN_ORDER} = \text{FALSE.} \end{cases} \quad (39)$$

Oxygen Dependence

To account for oxidation, three options are available depending on the parameters `IO2TYPE (IRXNS)` and `ORDERO2 (IRXNS)` (noted $n_{O2,k}$) as express in Equation (40). By default, `ORDERO2 (IRXNS) = 0` and `IO2TYPE = 1`, which means the reaction is not oxidative.

$$g_k(Y_{O2}) = \begin{cases} 1, & \text{if } n_{O2,k} = 0 \\ (1 + Y_{O2})^{n_{O2,k}} - 1, & \text{if } n_{O2,k} \neq 0 \text{ and } \text{IO2TYPE} = 0 \\ Y_{O2}^{n_{O2,k}} & \text{if } n_{O2,k} \neq 0 \text{ and } \text{IO2TYPE} = 1 \end{cases} \quad (40)$$

As a reminder, you should specify which gaseous species represents oxygen, as described in Section 10.2.

Critical Temperature

Each reaction can be assigned a critical temperature `TCRIT (IRXNS)`. When the local temperature is below this threshold, the reaction rate is set to zero:

$$r_k = 0 \quad \text{if} \quad T < T_{crit,k} \quad (41)$$

This feature is useful when modeling materials that undergo a significant change in properties at a specific temperature. For example, you can represent such behavior by defining two separate condensed species: one that exists below the critical temperature and another that exists above it, each with its own properties. An instantaneous reaction occurring at the critical temperature can then be used to switch from one species to the other.

11.5. Enthalpy of reaction

The enthalpy of reaction is a thermodynamic quantity that refers to the energy released or absorbed during a chemical transformation k under reference conditions. It is denoted by ΔH_k and expressed per unit mass ($\text{J} \cdot \text{kg}^{-1}$) or per mole.

In practice, defining ΔH_k is not straightforward. Experimental determination is often ambiguous because it is difficult to separate latent heat effects, the energy required for volatilization, and sensible heat changes. Even for a single polymer, reported values of reaction enthalpy can vary by an order of magnitude depending on how Differential Scanning Calorimetry (DSC) data are interpreted and which methodology is applied [ref]. Consequently, the pyrolysis modeling community lacks a unified and rigorous standard for defining ΔH_k .

To address this, Gpyro provides two approaches for computing the volumetric heat source term $\dot{Q}_k'''$ (in $\text{J} \cdot \text{m}^{-3} \cdot \text{s}^{-1}$) from the enthalpies of reaction:

11.5.1. Constant Heat of Reaction

The most common approach is to assume a constant enthalpy of reaction. This is achieved by setting `CONSTANT_DHVOL=.TRUE.` in the `&GPYRO_GENERAL` namelist (default option). Under this assumption, the volumetric heat source term is expressed as:

$$\dot{Q}_k''' = -\dot{\omega}_{fB,k}''' \Delta H_{sol,k} - \dot{\omega}_{fg,k}''' \Delta H_{vol,k} \quad (42)$$

where $\Delta H_{sol,k}$ and $\Delta H_{vol,k}$ represent the solid and volatile contributions to the enthalpy of reaction, specified by `DHS (IRXNS)` and `DHV (IRXNS)` respectively. The terms $\dot{\omega}_{fB,k}'''$ and $\dot{\omega}_{fg,k}'''$ are the production rates of condensed and gaseous products, while $\dot{\omega}_{dA,k}'''$ is the destruction rate of the reactant species (all in $\text{kg}\cdot\text{m}^{-3}\cdot\text{s}^{-1}$).

If $\Delta H_{vol,k} = \Delta H_{sol,k}$, this simplifies to:

$$\dot{Q}_k''' = -\dot{\omega}_{dA,k}''' \Delta H_{sol,k} \quad (43)$$

It is recommended to set these two values equal so that the heat of reaction is consistently defined per kilogram of the reactant species A consumed by reaction k .

Positive values of $\Delta H_{sol,k}$ and $\Delta H_{vol,k}$ correspond to **endothermic reactions**, while negative values indicate **exothermic reactions**. By default, `DHV (IRXNS) = DHS (IRXNS) = 0`.

11.5.2. Conservative Formulation of Heat of Reaction

A more rigorous approach ensures that the volumetric heat source term satisfies energy conservation. This is activated by setting `CONSTANT_DHVOL=.FALSE.` in the `&GPYRO_GENERAL` namelist. In this case, the source term is computed as:

$$\dot{Q}_k''' = -\dot{\omega}_{fB,k}''' \Delta H_{sol,k}^0 - \dot{\omega}_{fg,k}''' \Delta H_{vol,k}^0 - \dot{\omega}_{fg,k}''' (h_A(T) - h_g(T)) \quad (44)$$

Here, the user specifies `DHV (IRXNS)` as :

$$\Delta H_{vol,k}^0 = h_g^0 - h_A^0 \quad (45)$$

which is the difference in reference enthalpy between the gas and the condensed reactant, and `DHS (IRXNS)` :

$$\Delta H_{sol,k}^0 = h_B^0 - h_A^0 \quad (46)$$

which is the difference in reference enthalpy between the condensed product and the condensed reactant.

This formulation guarantees that the energy balance accounts for both chemical enthalpy changes and sensible enthalpy differences, as detailed in the Technical Guide (see Chapter 3.5).

The main challenge with this approach lies in determining accurate reference enthalpies, which are sometimes available for simple gases [ref] but are often difficult to characterize for complex condensed phases. Furthermore, when multiple reactions and steps are involved, the definitions

of $\Delta H_{vol,k}^0$ and $\Delta H_{sol,k}^0$ must remain consistent across all reactions to ensure a coherent reference enthalpy framework for all species.

11.6. Example

Here, an example is provided to illustrate how to define a condensed-phase reactions. It is important to note that the following is only a partial input file, where only the parameters necessary to define the reactions are explicitly specified. Although the procedure for declaring reactions in Gpyro may initially seem complex, this example demonstrates that, in practice, the process is straightforward.

```
&GPYRO_GENERAL
  CONSTANT_DHVOL = .TRUE. ! Default option
  CONVENTIONAL_RXN_ORDER = .FALSE. ! Default option
  ...
/

&GPYRO_SPROPS  NSSPEC = 4
  NAME(1) = 'foam_virgin', R0(1) = 2200.0, ...
  NAME(2) = 'foam_critic', R0(2) = 2000.0, ...
  NAME(3) = 'char',          R0(3) = 400.0, ...
  NAME(4) = 'ash',           R0(4) = 200.0, ...
/

&GPYRO_RXNS NRXNS = 4

!Reaction 1 : critical transition
  CFROM(1) = 'foam_virgin', CTO(1) = 'foam_critic'
  E(1) = 0.0, A(1) = 1.0E+18
  TCRIT(1) = 500.0
  CHI(1) = 0.0

!Reaction 2 : decomposition into char + CO
  CFROM(2) = 'foam_critic', CTO(2) = 'char'
  E(2) = 220.0, A(2) = 1.0E+12
  CHI(2) = 0.5

!Reaction 3 : oxidation of char into ash + CO
  CFROM(3) = 'char', CTO(3) = 'ash'
  E(3) = 180.0, A(3) = 5.0E+10
  ORDER(3) = 3, IO2TYPE(3) = 1, ORDERO2(3) = 1
  CHI(3) = 1.0
  DHS(3) = -1.0E+5, DHV(3) = -1.0E+5

!Reaction 4 : volatilization of char into HCN
  CFROM(4) = 'char', CTO(4) = 'gases'
  E(4) = 300.0, A(4) = 2.0E+11
  DHS(4) = 1.0E+5, DHV(4) = 1.0E+5
/

&GPYRO_GPROPS  NGSPEC = 3
  IO2 = 1
  NAME(1) = 'O2'
```

```

NAME (2) = 'CO'
NAME (3) = 'HCN'
/

&GPYRO_GYIELDS
! Reaction 1 : foam_virgin → foam_critic (no gases, force sum = 1.0)
GYIELDS (1,1) = 1.0

! Reaction 2 : foam_critic → char + CO
GYIELDS (2,2) = 1.0

! Reaction 3 : char + O2 → ash + CO
GYIELDS (1,3) = -1.0    ! oxygen consumed
GYIELDS (2,3) = 2.0    ! CO produced

! Reaction 4 : char → gases (HCN)
GYIELDS (3,4) = 1.0
/

```

First reaction

The first reaction introduces a critical transition. Below 500 K, the material exists as *foam_virgin*, and once the temperature reaches 500 K it instantaneously transforms into *foam_critic*. Since $\chi = 0$, the stoichiometric coefficient is $\theta_1 = 1$, which means the transformation is purely solid-to-solid without mass loss. The density change, however, induces a small deformation:

$$\mathfrak{D}_1 = \frac{\rho_{virgin}}{\rho_{critic}} \theta_1 = \frac{2200}{2000} = 1.1$$

Thus, the volume expands by about 10%. No gases are released, but to avoid an error message, GYIELDS (1, 1) is set to 1.0.

Second reaction

The second reaction models the decomposition of *foam_critic* into *char* and carbon monoxide. With $\chi_2 = 0.5$, the stoichiometric coefficient becomes

$$\theta_2 = 1 + \left(\frac{\rho_{char}}{\rho_{critic}} - 1 \right) \chi_2 = 60\%$$

The associated deformation is

$$\mathfrak{D}_2 = \frac{\rho_{critic}}{\rho_{char}} \theta_2 = 3$$

Since neither IKINETICMODEL (2) nor ORDER (2) are specified, the default kinetic model applies: a first-order thermal decomposition reaction. The reaction rate is therefore expressed as:

$$r_2 = \rho Y_{critic} Z_2 \exp \left(-\frac{E_2}{RT} \right)$$

Finally, since neither DHS (2) nor DHV (2) are defined, the heat of reaction is assumed to be zero.

Third reaction

The third reaction describes the oxidation of *char* into *ash* and carbon monoxide. In this case, oxygen is explicitly consumed, with GYIELDS (1, 3) = -1.0, while two CO molecules are produced, with GYIELDS (1, 3) = 2.0. The oxidative behavior is controlled by IO2TYPE (3) =

1 and $ORDERO2(3)=1$, indicating first-order dependence on oxygen concentration. . Note that this will not occur unless oxygen is present.

The stoichiometric coefficient of the condensed phase is calculated as:

$$\theta_3 = 1 + \left(\frac{\rho_{ash}}{\rho_{char}} - 1 \right) \chi_3 = 50\%$$

Since $CHI(3)=1$, the reaction corresponds to a charring process without net deformation.

The negative enthalpy values ($DHS(3)$ and $DHV(3)$) specify that the oxidation is exothermic. The reaction follows an nth-order model with order three for the condensed species, combined with a first-order dependence on oxygen. The reaction rate is therefore expressed as:

$$r_3 = (1 - \alpha_{char})^2 \rho Y_{char} Y_{O_2} Z_3 \exp\left(-\frac{E_3}{RT}\right)$$

Fourth reaction

Finally, the fourth reaction represents a non-charring process, where *char* volatilizes entirely into HCN. Because $CTO(4) = \text{"gases"}$, no condensed product remains and $\theta_4 = 0$. The enthalpy values defined here are positive, indicating that this is an endothermic process.

12.0 HOMOGENEOUS GAS-PHASE REACTIONS

In addition to heterogeneous reactions involving the condensed and gaseous phases, the user may also specify any number of homogeneous gas phase reactions. Such reactions can be used to account for tar cracking where large hydrocarbon molecules are broken into smaller hydrocarbon molecules or the oxidation of pyrolysate as it flows through a char layer toward the surface where both the oxygen concentration and temperature are highest.

For convenience, a summary of all input parameters related to gas-phase reactions is provided in Table XX of the Appendix.

12.1. Declaring Gas-Phase Reactions

The total number of homogeneous gas-phase reactions is defined in the namelist &GPYRO_HGRXNS through the keyword NHGRXNS. Each reaction is identified by its index IGRXNS, ranging from 1 to NHGRXNS.

In Gpyro, the general formulation of a gas-phase reaction l is written as:



Here, A_l and B_l are the gaseous reactants, while G_j represents the gaseous species j . The coefficients $y_{g,j,l}$ form the *homogeneous gaseous species yield matrix*, analogous to the condensed-phase yield matrix described in Section 11.3.

A species G_j is considered a product if $y_{g,j,l} > 0$. If $y_{g,j,l} = 0$, the species does not participate in the reaction, and if $y_{g,j,l} < 0$, the species acts as a reactant. This formulation allows for multiple reactants to be specified in a single gas-phase reaction.

However, for practical reasons, Gpyro isolates two primary reactants (A_l and B_l) that are explicitly declared by the user. Their names are given through CREATANT1 (IGRXNS) and CREATANT2 (IGRXNS). This structure makes it possible to directly control the reaction rate law as a function of the concentrations of two designated species, while any additional reactants or products are accounted for through the yield matrix. Both names are case-sensitive and must match those defined in &GPYRO_GPROPS.

The yield coefficients $y_{g,j,l}$ are then defined in the namelist &GPYRO_HGYIELDS through the parameter HGYIELDS (IGAS, IGRXNS). By default, all entries are zero. To ensure correct stoichiometry, the entry corresponding to A_l must be set to -1 , and the sum over all gaseous species of HGYIELDS (: , IGRXNS) must equal zero.

12.2. Chemical Kinetics

The rate of a homogeneous gas-phase reaction l is expressed as:

$$r_l = \bar{\psi}[A_l]^{p_l}[B_l]^{q_l}T^{b_l}Z_l \exp\left(-\frac{E_l}{RT_g}\right) \quad (48)$$

Where:

- $\bar{\psi}$ is the porosity (See Section 9.3.1),
- $[A_l]$ and $[B_l]$ are the molar concentrations of the two reactants,
 p_l and q_l are the partial reaction orders, specified via P (IGRXNS) and Q (IGRXNS),
- b_l is an empirical temperature exponent, defined by B (IGRXNS),
- Z_l is the pre-exponential factor (s^{-1}), given by Z (IGRXNS),
- E_l is the activation energy ($kJ \cdot mol^{-1}$), defined by E (IGRXNS),
- R is the universal gas constant,
- T_g is the local gas temperature in Kelvin.

12.3. Heat of Reaction

The heat of a gas-phase reaction, per unit mass of reactant 1 consumed ($J \cdot kg^{-1}$), is specified with DH (IGRXNS). By convention, DH is negative for exothermic reactions and positive for endothermic reactions. If not specified, the default value is zero.

12.4. Example

The following is an example of &GPYRO_HGRXNS and &GPYRO_HGYIELDS line:

```
&GPYRO_HGRXNS  NHGRXNS      = 1
                  CREACTION1(1) = 'pyrolysate'
                  CREACTION2(1) = 'oxygen'
                  P              (1) = 1.0
                  Q              (1) = 1.0
                  Z              (1) = 1D10
                  E              (1) = 195.0
                  DH              (1) = -15D6 /

&GPYRO_HGYIELDS HGYIELDS(1,1) = -1.0
                  HGYIELDS(2,1) = -3.0
                  HGYIELDS(3,1) = 0.0
                  HGYIELDS(4,1) = 4.0 /
```

This input specifies that reaction 1 consumes 1 kg of gaseous species 1 ('pyrolysate') and 3 kg of gaseous species 2 ('oxygen') and produces 4 kg of gaseous species 4 ('products'). Species 3 does not participate in the reaction. The stoichiometry is enforced through the yield matrix, where negative coefficients correspond to consumed reactants and positive coefficients to produced species. From the kinetic definition, this is a second-order reaction, with partial orders $p_l = 1$ for pyrolysate and $q_l = 1$ for oxygen. The enthalpy of reaction is negative, meaning that the reaction is exothermic.

13.0 OUTPUT FILES

In Gpyro, the dump routines allow users to specify which simulation data should be saved during the run. These settings are configured via the `&GPYRO_OUTPUT` line, and all output files are named using the string defined by the `CASENAME` keyword.

The available output types are organized into three main categories:

- **Point quantities** (values at specific locations),
- **Profiles** (spatial distributions in a specific direction),
- **Smokeview files**, used to visualize 2D/3D simulation results [ref].

Details of each output type are provided in Sections 0 to 13.4. The complete list of supported output quantities is available in Table 4. Note that not all quantities can be exported for every output type, limitations are indicated in the tables.

It is also possible to control the output frequency to capture rapid phenomena such as ignition or flame spread. This functionality is described in Section 13.5.

In addition to the standard output types, Gpyro generates several auxiliary files for diagnostics, performance monitoring, and debugging, as described in Section 13.6.

13.1. Available quantities to dump

A summary of all available input parameters for the `&GPYRO_OUTPUT` namelist is provided in Table 17.

Table 4: Quantities available for dump.

Name	Index meaning	Description	Units
'TEMPERATURE'		Condensed-phase temperature	°C
'ENTHALPY'		Condensed-phase enthalpy	J/kg
'YI'	ISSPEC	Mass fraction of condensed-phase species	kg/kg
'XI'	ISSPEC	Volume fraction of condensed-phase species	m ³ /m ³
'CI'	ISSPEC	Concentration of condensed-phase species	kg/m ³
'YISUM'		Sum of all condensed-phase mass fractions	kg/kg
'REACTION_RATE_K'	IRXN	Rate of k th condensed-phase reaction	kg/m ³ -s
'YJ'	IGSPEC	Mass fraction of gaseous species	kg/kg
'CJ'	IGSPEC	Concentration of gaseous species	kg/m ³
'YJSUM'		Sum of all gas-phase mass fractions	kg/kg
'REACTION_RATE_L'	IHGRXN	Rate of ℓ^{th} homogeneous gaseous reaction	kg/m ³ -s
'S'		Source term in condensed-phase energy eq.	J/m ³ -s
'QSC'		Source term induces by chemical reaction	J/m ³ -s
'THERMAL_CONDUCTIVITY_Z'		Effective thermal conductivity in z-direction	W/m-K
'THERMAL_CONDUCTIVITY_X'		Effective thermal conductivity in x-direction	W/m-K
'THERMAL_CONDUCTIVITY_Y'		Effective thermal conductivity in y-direction	W/m-K
'BULK_DENSITY'		Condensed-phase bulk density	kg/m ³
'SOLID_DENSITY'		Condensed-phase pure solid density	kg/m ³
'SPECIFIC_HEAT_CAPACITY'		Condensed-phase specific heat capacity	J/kg-K
'DLTZN'		Grid size in z direction	m
'DLTXN'		Grid size in y direction	m
'DLTYN'		Grid size in x direction	m

'PRESSURE'		Overpressure	Pa
'MASS_FLUX_TOTAL_Z'		z-direction mass flux	g/m ² -s
'MASS_FLUX_TOTAL_X'		x-direction mass flux	g/m ² -s
'MASS_FLUX_TOTAL_Y'		y-direction mass flux	g/m ² -s
'GAS_TEMPERATURE'		Gas-phase temperature	°C
'GAS_ENTHALPY'		Gas-phase enthalpy	J/kg-K
'GAS_DENSITY'		Gas-phase density	kg/m ³
'TG-T'		Gas temperature minus solid temperature	°C
'SHYI'		SHYI correction coefficient	J/m ³ -s
'PERMEABILITY_Z'		z-direction Permeability	m ²
'PERMEABILITY_X'		x-direction Permeability	m ²
'PERMEABILITY_Y'		y-direction Permeability	m ²
'POROSITY'		Porosity	m ³ /m ³
'D12'		Gaseous diffusion coefficient	m ² /s
'SHGP'		Positive part of gas energy eq. source term	J/m ³ -s
'SHGM'		Negative part of gas energy eq. source term	J/m ³ -s
'QSG'		Rate of heat transfer from solid to gas phase	J/m ³ -s
'RYIDZSIGMA'	ISPEC	$(\bar{\rho}Y_i\Delta z)_\Sigma$	kg/m ²
'UNREACTEDNESS'	ISPEC	1 - α	-
'GOMEGA3'		Conversion rate of solid to gas	kg/m ³ -s
'GGR' ³	IGSPEC	Gas generation rate	kg/m ³ -s
'RE'		Reynolds number	-
'NU'		Nusselt number	-
'HCV'		Volumetric heat transfer coefficient	W/m ³ -K
'M/MO'		Mass divided by initial mass in a cell	kg/kg
'TOTAL_MASS' ³		Total mass in the mesh	kg
'MLR' ³	IGSPEC ¹	Flux of gaseous species	⁴
'MPPI' ^{2,3}	ISPEC	Mass per unit area of condensed species	kg/m ²
'THICKNESS' ^{2,3}		Total thickness	m
'DT' ^{2,3}		Timestep	s
'N_ITERATIONS' ^{2,3}		Number of iterations for convergence	-

ISSPEC: Index of solid species

IGSPEC: Index of gaseous species

IRXN: Index of heterogeneous/condensed-phase reaction

IHGRXN: Index of heterogeneous gaseous reaction

¹ Specify IGSPEC = 0 for total MLR.

² Does not vary spatially

³ Available only as a point quantity

⁴ The unit of MLR depends on the simulation dimensionality. In 1D, it is expressed in g·m⁻²·s⁻¹, in 2D in g·m⁻¹·s⁻¹, and in 3D in g·s⁻¹. For 0D simulations (TGA), it is defined as the total mass of the sample divided by the initial mass, multiplied by 1000 and expressed per second. This unconventional unit is kept for backward compatibility.

13.2. Point dumps

13.2.1. Configuration

Several quantities can be exported as point values, as listed in Table 4. To activate this feature, the total number of point quantities must be specified using the keyword N_POINT_QUANTITIES.

For each quantity I (with I ranging from 1 to N_POINT_QUANTITIES), the following parameters may be defined:

- POINT_QUANTITY(I) : name of the quantity to dump,
- POINT_Z(I) : vertical coordinate of the point (in meters, with 0 at the upper surface),
- POINT_X(I) : optional x coordinates, required only for 2D or 3D simulations,

- `POINT_Y(I)`: optional y coordinates, required only for 3D simulations,
- `POINT_IMESH(I)`: optional mesh identifier; required when using multiple meshes to specify which mesh the point belongs to, If `POINT_IMESH(I)=0`, the quantity will be dumped for all meshes.
- `POINT_QUANTITY_INDEX(I)`: optional index, used for quantities requiring an additional identifier (e.g., species index for mass fractions ('YI')).

Point quantities follow a **Lagrangian approach**: each point is associated with a computational cell at the start of the simulation. During the simulation, the geometry may deform (due to swelling, shrinkage, etc.), but the output always refers to the initial location of the assigned cell. Coordinates defined by the user are thus interpreted relative to the initial, undeformed mesh.

Some quantities can be requested via the point-dump mechanism even though they are not spatially localized. Examples include the mass loss rate ('MLR') or total sample mass ('TOTAL_MASS'). For such quantities, spatial coordinates (and `POINT_Z(I)`, `POINT_X(I)`, and `POINT_Y(I)`) are not required and can be omitted.

The output frequency is controlled by the `DTDUMP_POINT` parameter, which sets the time interval (in seconds) between successive dumps.

13.2.2. Example

The configuration below shows how to request five point quantities:

```
&GPYRO_OUTPUT N_POINT_QUANTITIES      = 5
               DTDUMP_POINT             = 0.1
               POINT_QUANTITY(1)        = 'TEMPERATURE'
               POINT_Z(1)                = 0.0
               POINT_QUANTITY(2)        = 'YI'
               POINT_QUANTITY_INDEX(2)  = 1
               POINT_Z(2)                = 0.01
               POINT_QUANTITY(3)        = 'YI'
               POINT_QUANTITY_INDEX(3)  = 2
               POINT_Z(3)                = 0.01
               POINT_QUANTITY(4)        = 'MLR'
               POINT_QUANTITY_INDEX(4)  = 0
               POINT_QUANTITY(5)        = 'MLR'
               POINT_QUANTITY_INDEX(5)  = 1
               .
               .
               .
```

In this example, a total of five quantities are dumped every 0.1 seconds. The first is the top surface temperature at $z = 0.0$ m. The second and third are the condensed-phase mass fractions of species 1 and 2, extracted at 10 mm depth. The fourth requests the total mass loss rate (MLR), since the used index is 0. The fifth requests the mass loss rate of species 1 (using index 1).

13.2.3. Output File Format

Point quantities are written to a CSV file named using the value provided by the `CASENAME` parameter, with the suffix `_summary.csv`.

The first row of the file contains the column headers:

- The first column corresponds to time,
- The following columns correspond to the point quantities, listed in the order defined by their index `I` in the input (`POINT_QUANTITY(1)`, `POINT_QUANTITY(2)`, etc.).

Each subsequent row contains the data values at a given output time, as defined by `DTDUMP_POINT`.

13.3. Profile dumps

Profile dumps allow the user to export the spatial distribution of a quantity along a given axis (x, y, or z) at specified locations in the remaining orthogonal directions. This functionality is compatible with 1D, 2D, and 3D configurations.

13.3.1. Configuration

Quantities to dump

The number of profile quantities to output is defined using `N_PROFILE_QUANTITIES` parameter. Each profile `J` (from 1 to `N_PROFILE_QUANTITIES`) to be dumped is defined by the parameter `PROFILE_QUANTITY(J)`, which specifies the quantity to be extracted (see Table 4). It should be noted that not all quantities listed in Table 4 are available for profiles, as indicated by Note 3.

As with point quantities, `PROFILE_QUANTITY_INDEX(J)` is an optional parameter used to specify the index when needed, particularly for species-related quantities.

Profile Direction and Positioning

The profile orientation is specified using `PROFILE_DIRECTION(J)`, while its spatial location depends on the dimensionality of the simulation and is defined by `PROFILE_COORD1(J)` and `PROFILE_COORD2(J)`.

In 1D simulations: no coordinate needs to be specified, as the domain is inherently one-dimensional.

In 2D simulations : only `PROFILE_COORD1(J)` is required. It defines the position along the axis perpendicular to the profile direction. For example, if `PROFILE_DIRECTION = 'z'`, the profile is extracted along the z-axis at a fixed x-coordinate (`PROFILE_COORD1(J)`).

In 3D simulations: both `PROFILE_COORD1(J)` and `PROFILE_COORD2(J)` are required. These define the fixed positions in the two directions orthogonal to the profile direction.

If `PROFILE_DIRECTION(J) = 'x'`, the profile is taken along the x-axis at constant `y = PROFILE_COORD1(J)` and `z = PROFILE_COORD2(J)`. If `PROFILE_DIRECTION(J) = 'y'`, the profile is taken along the y-axis at constant `x = PROFILE_COORD1(J)` and `z = PROFILE_COORD2(J)`. If `PROFILE_DIRECTION(J) = 'z'`, the profile is taken along the z-axis at constant `x = PROFILE_COORD1(J)` and `y = PROFILE_COORD2(J)`.

Additionally, if multiple meshes are defined, the optional parameter `PROFILE_IMESH(J)` can be used to specify from which mesh the profile should be extracted. This parameter behaves similarly to `POINT_IMESH(J)` in point dumps.

Spatial and Temporal Sampling

The parameter `DTDUMP_PROFILE` controls the time interval (in seconds) at which all profiles are dumped.

The spatial sampling frequency for each profile can be adjusted individually using the optional parameter `PROFILE_ISKIP(J)`. This parameter specifies how often values are recorded along the profile line, for example, a value of 5 records data from every fifth cell. By default, if `PROFILE_ISKIP(J)` is not specified, data is recorded at every cell. This feature is useful for finely meshed domains or to reduce output file size.

13.3.2. Example

The input block below defines three profile outputs using the `&GPYRO_OUTPUT` namelist. This configuration assumes a 3D simulation, as all profiles specify both `PROFILE_COORD1` and `PROFILE_COORD2` to locate the extraction planes in space.

```
&GPYRO_OUTPUT N_PROFILE_QUANTITIES = 3
               DTDUMP_PROFILE       = 1.0

               PROFILE_QUANTITY(1)   = 'TEMPERATURE'
               PROFILE_DIRECTION(1)  = 'z'
               PROFILE_COORD1(1)     = 0.10
               PROFILE_COORD2(1)     = 0.02
               PROFILE_ISKIP(1)      = 10

               PROFILE_QUANTITY(2)   = 'TEMPERATURE'
               PROFILE_DIRECTION(2)  = 'x'
               PROFILE_COORD1(2)     = 0.02
               PROFILE_COORD2(2)     = 0.20

               PROFILE_QUANTITY(3)   = 'XI'
               PROFILE_QUANTITY_INDEX(3) = 1
               PROFILE_DIRECTION(3)  = 'y'
               PROFILE_COORD1(3)     = 0.02
               PROFILE_COORD2(3)     = 0.05/
```

This input defines three profiles to be dumped every 1.0 second as controlled by the `DTDUMP_PROFILE` parameter.

Profile 1 extracts temperature along the z-direction at fixed positions $x = 0.10$ m and $y = 0.02$ m. The spatial sampling is defined by `PROFILE_ISKIP(1) = 10`, which means one out of every ten cells along the z-axis is recorded.

Profile 2 also extracts temperature, this time along the x-direction, again at fixed $y = 0.02$ m and $z = 0.20$ m. No `PROFILE_ISKIP` is specified here, so the default behavior applies: every cell along the x-axis is recorded.

Profile 3 extracts the mass fraction (XI) for species index 1, along the y-direction, at $x = 0.02$ m, $z = 0.05$ m.

13.3.3. Output File Format

For each defined profile, a separate CSV file is generated. The filename includes the case name (CASENAME) followed by a suffix indicating the mesh index, profile index, and the dumped quantity. The naming convention is as follows:

`casename_profile_IMESH_J_QUANTITY(INDEX).csv`

For example, for mesh index 1, profile index 3, and quantity XI with species index 1, the file name will be:

`CASENAME_profile_01_02_YI(01).csv`

The first row of each file contains the initial coordinates of the sampled points along the specified profile direction (`PROFILE_DIRECTION`). These coordinates correspond to the cell centers and are determined based on the mesh structure and the optional spatial skip parameter (`PROFILE_ISKIP`). Each subsequent row represents a time step and begins with the simulation time (in seconds), followed by the values of the selected quantity at each sampled point.

It is important to note that the coordinates in the header represent the initial (undeformed) geometry of the domain. If material deformation occurs during the simulation, these coordinates no longer reflect the actual positions of the profile points. To reconstruct the true positions over time, the 'DLTZN' quantity can be output as an additional profile. This quantity provides the local cell height at each time step. By accumulating these local thickness values (e.g., summing them from the bottom of the profile), it becomes possible to reconstruct the deformed spatial coordinate of each point and accurately plot quantities as a function of the evolving geometry.

13.4. Smokeview visualization

For 2D and 3D simulations only, Gpyro can generate output files compatible with NIST Smokeview, enabling graphical visualization of the simulation results. These files allow users to view transient 2D slices of various quantities such as temperature, density, or thermal conductivity directly within Smokeview.

13.4.1. Configuration

The number of quantities to export for Smokeview visualization is specified using the parameter `N_SMOKEVIEW_QUANTITIES`. Each quantity `I` must be configured using the following input parameters:

- `SMOKEVIEW_QUANTITY(K)` : Name of the quantity to visualize (see Table 4),
- `SMOKEVIEW_QUANTITY_INDEX(K)` : (Optional) Index used for species-specific quantities,
- `SMOKEVIEW_IMESH(K)` : (Optional) Mesh index if multiple mesh exist,
- `SMOKEVIEW_PLANE(K)` : Specifies the orientation of the slice plane. Acceptable values are 'xy', 'xz', or 'yz'.
- `SMOKEVIEW_LOCATION(K)` : Location along the axis perpendicular to the chosen plane, where the slice will be taken.

These inputs follow the same logic as for profile quantities. As with profile outputs, not all entries listed in Table 4 are available for Smokeview visualization; this limitation is indicated by note 3 in Table 4 .

13.4.2. Example

Below is a configuration example that defines a Smokeview slice of the total reaction rate (`REACTION_RATE_K`) in the yz plane at $x = 0.05$, using mesh 1, with a dump interval defined every 1 second:

```
&GPYRO_OUTPUT N_SMOKEVIEW_QUANTITIES      = 1
               DTDUMP_SMOKEVIEW            = 1.0
               SMOKEVIEW_QUANTITY(1)       = 'REACTION_RATE_K'
               SMOKEVIEW_QUANTITY_INDEX(1) = 0
               SMOKEVIEW_IMESH(1)          = 1
               SMOKEVIEW_PLANE(1)          = 'yz'
               SMOKEVIEW_LOCATION(1)       = 0.05/
```

13.4.3. Output File Format

Gpyro generates a main visualization file named `CASENAME.smv`. This file serves as the entry point and can be directly opened with Smokeview. The associated slice data are stored in binary `.sf` files with the following naming convention: `CASENAME_XX.sf`, where `XX` corresponds to the index of the dumped quantity.

For detailed usage and visualization options, refer to the official Smokeview documentation [ref].

13.5. Changing the dump interval mid-simulation

In some situations, it may be desirable to change the interval at which Gpyro writes to disk. For example, the time scale associated with smolder propagation is much longer than the timescale

associated with transition from smolder to flame. Consequently, disk dumps at an interval of 1 s may be appropriate during smolder propagation, but a smaller dump interval (e.g., 0.001 s) would be needed to “see” the transition to flame. For this reason, an option was added wherein the dump interval can be reduced when the temperature (solid or gas) increases above a certain value. More specifically, when the gas or solid temperature increases above `TMP_REDUCED_DTDUMP` (units of K), all output files are written to disk at an interval of `REDUCED_DTDUMP` (units of s).

13.6. Other Output Files

In addition to profile and Smokeview outputs, Gpyro generates several auxiliary files that can be useful for monitoring, debugging, and performance evaluation.

13.6.1. .out File

Gpyro automatically creates a text file named `CASENAME.out`. This file provides a complete record of all user-defined input parameters as interpreted by the solver. It is particularly useful for verifying that no input errors (e.g., typos in namelist groups or parameters) are present and that the configuration matches the user’s intention.

In addition to listing inputs, the `.out` file includes runtime summaries of key quantities, which allow users to monitor the progress and stability of the simulation.

13.6.2. CPU Timing File

Upon successful completion of a simulation, Gpyro generates a file named `CASENAME_timing.csv`. This file details the CPU time consumed by the major subroutines. While primarily intended for debugging and performance optimization, this information can also help identify bottlenecks in complex simulations.

13.6.3. Iteration Convergence File

If the optional parameter `DUMP_DETAILED_CONVERGENCE = .TRUE.` is set, Gpyro outputs a file named `iterations.csv`. This file logs the number of iterations performed by each solver at every time step. It is a valuable resource for debugging convergence issues or analyzing the numerical behavior of the solution process.

14.0 MATERIAL PROPERTY ESTIMATION

Gpyro is distributed with a companion material property estimation program (see Section 2.5) that is based on genetic algorithm optimization. It is recommended that the spreadsheet-based front end be used when running the material property estimation program because it is easy to make mistakes when manually setting up text files due to the (usually) large number of parameters to be optimized. Therefore, the discussion in this section is focused around the spreadsheet-based front end rather than the actual Namelist groups and keywords that are written to an input file by the spreadsheet-based front end.

All worksheets necessary to use the genetic algorithm property estimation program are contained in the spreadsheet-based front end. There are three worksheets where the user specifies relevant parameters for the property estimation process: GA_GenInput, GA_phi, and GA_Vars. Each of these worksheets is explained below. Additionally, there are a number of worksheets in which experimental data must be placed. They are named 01, 02, 03 for experiment 1, 2, 3, and so on. The experimental data in these sheets must be laid out in a specific way, as will be described in detail below.

The user is cautioned that there is even less error checking in the Gpyro property estimation program than in the standalone Gpyro program and the inputs are somewhat more cryptic than for the standalone Gpyro program. Thus, it is easy to make a mistake during data entry, and the user should carefully check the model output to make sure the code is doing what the user intended. Often, runtime errors result from erroneous data entry, but the actual cause can be difficult to track down, even when the code is run from within a debugger.

The Gpyro property estimation program can be run either in serial or in parallel, and the serial executable file is the same as the parallel executable file. Parallelization is achieved using Message Passing Interface. The Gpyro property estimation program has been successfully run in parallel on more than 100 cores.

The following command executes the Gpyro property estimation program on a single processor:

```
gpyro_propest
```

Since the input file name must be `gpyro.data`, it is not necessary to specify an input file name as with the standalone Gpyro program. The name of the input file is hardcoded to avoid difficulties encountered with MPI i/o.

The parallel version has not been tested under Windows, but under Linux it is launched by issuing a command similar to the following:

```
mpirun -np 32 -machinefile machines.linux -machinedir ./  
/home3/clauten/bin/gpyro_propest &
```

The syntax may be slightly different with LAM MPI instead of MPICH MPI, but this example tells the code to run on 32 cores (`-np 32`), look up the machine names to run on in the file `macines.linux` (`-machinefile machines.linux`) located in the current directory (`-machinedir ./`), and execute the binary file called `gpyro_propest` located in the `/home3/clauten/bin/`.

The Gpyro property estimation program can be halted gracefully (important when running in parallel across multiple nodes) by creating a file called “stop.stop” in the working directory. The Gpyro property estimation program only checks for the existence of this file at the completion of a generation, so it may take several minutes (or even several hours) to shutdown after the file stop.stop is created. The contents of stop.stop are irrelevant; the program merely checks to see if it exists and shuts down gracefully if it does.

14.1. Format of input files for material property estimation program

The input file for the material property estimation program is the same as for the standalone Gpyro input file discussed above, plus three additional Namelist groups. Whereas the input file for the standalone Gpyro program can have any file name, the input file for the material property estimation program must be named `gpyro.data` (this simplifies running in parallel with MPI). The three additional Namelist groups that must be specified in this input file (and the analogous worksheet name in the spreadsheet-based front end) are listed in Table 5.

Table 5: Gpyro property estimation Fortran Namelist groups, associated worksheet in spreadsheet-based front end, and description of Namelist group entries.

Namelist group name	Worksheet name	Description
&GA_GENINPUT	GA_GenInput	General/miscellaneous parameters
&GA_PHI	GA_Phi	Fitness weightings
&GA_VARS	GA_Vars	Variables to be optimized

An additional input file for each experimental measurement to be used in the property estimation process is required. The naming convention for these files is `exp_xx_yy.csv` where `xx` corresponds to IPHI (more on this in the fitness weightings section) and `yy` is ICASE (the index corresponding to the case number being simulated). The format of these input files is as follows:

```
NDT
t1, y1
t2, y2
t3, y3
.
.
.
```

where t_n is the n^{th} time point and y_n is the experimental quantity (mass loss rate, temperature, or thickness) at that time. Mass loss rates must be specified in units of $\text{g/m}^2\text{-s}$, temperatures in degrees

Celsius, and thickness in meters. For example, the following is a valid experimental data file containing five temperature points:

```
5
0.0, 27.0
5.0, 50.0
30.0, 200.0
60.0, 400.0
500.0, 450.0
```

14.2. Miscellaneous parameters – GA_GenInput worksheet

General parameters related to the genetic algorithm are specified in the GA_GenInput worksheet. The parameter `NGEN` is the number of generations, or evolution steps, that will be taken (unless the user halts execution prematurely). `NINDIV` is the number of individuals (or chromosomes) in a population. These concepts are explained in Section 7.2.2 of the Gpyro Technical Reference [1]. The parameter `MAXCOPIES` is the target (maximum) selection number described in Section 7.2.3 of the Gpyro Technical Reference [1].

If the code is run under Linux, then set `LINUX = .TRUE..` This parameter isn't very important, all it does is delete the stop file which tells the code to shutdown gracefully by issuing the shell command “rm -f stop.stop” under Linux and “del stop.stop” under DOS.

The parameter `SIMULATED_EXPERIMENTAL_DATA` should normally be set to `.FALSE..` If it is set to `.TRUE.,` then the Gpyro property estimation program generates a synthetic set of experimental data (instead of reading in actual experimental data) using the material properties specified elsewhere. This was used during the development process to evaluate how well the genetic algorithm could perform for idealized scenarios, i.e. infinitely accurate experimental measurements with a computational model that is a perfect representation of the physical processes occurring.

`FITMIN` and `FITCLIP` are two parameters that come in to play only when an individual has a very bad fitness, or the solution does not converge. Normally, these are set to zero. The variable `FITEXPONENT` is the parameter ζ in Equation 196 of the Gpyro Technical Reference [1]. The last two parameters affect reproduction (whereby offspring are generated from parents). Normally, offspring are generated as linear combinations of parents (Equation 201 of the Gpyro Technical Reference [1]). However, under some circumstances it may be useful to use the value of only one of the parents, i.e. an entire gene is used. This is analogous to setting r_j^i in Equation 201 of the Gpyro Technical Reference [1] to 1.0. The fraction of genes for which this is done is `WHOLEGENEFRACTION`.

The parameter `BRUTE_FORCE` indicates whether genes for the offspring should be generated randomly, rather than through the normal evolution process. Setting this parameter to `.TRUE.` essentially disables the genetic algorithm's ability to exploit promising solutions and the code reverts to a brute force search technique.

During the search process, some solutions that do not converge for the specified initial time step may be encountered. The code can either reduce the time step until a converged solution is obtained (as would be done for a standalone simulation) or immediately kill the trial solution that did not converge. This is controlled by the parameter `KILL_NONCONVERGED_SOLUTIONS`. It is recommended that this parameter be set to `.TRUE.` because if it is set to `.FALSE.`, then some simulations that do not converge could require a very small time step and consume a large amount of CPU time, while other processors sit idle (if running in parallel)

14.3. Fitness metric weightings – GA_phi worksheet

The fitness metric weightings, i.e. the ϕ values in Equation 197 of the Gpyro Technical Reference [1], are specified in the worksheet `GA_phi`. Each ϕ value multiplies a fitness function (Equation 196 of the Gpyro Technical Reference [1]) that quantifies the level of agreement between model predictions and experimental data. In this way, certain experiments or measurements can be given greater importance than others. The user should first specify the number of ϕ values (the number of experimental measurements) in cell A1, labeled `NGA_PHI`.

In addition to the actual ϕ value (specified in column F) a few additional parameters must be given. The integer `IQE` (index of `qe`) in column B corresponds to the case # listed in the `qe` worksheet. For example, if `IQE` = 2, then the ϕ value specified is for case #2 in the `qe` worksheet.

Several types of experimental data can be used as fitness metrics: cumulative mass loss, mass loss rate, temperature, and thickness. The code is told what type of experimental data the ϕ value corresponds to in column C (`TYPE`). Here, the user enters one of four 3-character strings to indicate which type experimental data is being used: `TMP` (temperature, °C), `MLR` (mass loss rate, g/m²-s), `DLT` (delta, or thickness, m), and `CML` (cumulative mass loss, g/m²).

Column D is labeled "`zT`". This column tells the code the depth (location) of a particular temperature measurement. For example, `zT` = 0 indicates a surface temperature measurement, `zT` = 0.005 indicates an in-depth temperature measurement 5 mm below the surface, and so on. Column D has meaning only if `TMP` is specified in column C.

Column E is labeled `tstop`. This is a time in seconds that can be less than or equal to the simulation time specified in the `qe` worksheet. Setting a value of `tstop` less than the simulation time directs Gpyro to calculate the fitness metric only through a time of `tstop`. This is convenient if the surface temperature was measured but deemed accurate only prior to ignition (at which point flaming flame can add uncertainty to temperatures measured via thermocouple or optical methods). By setting the value of `tstop` for the ϕ value corresponding to the surface temperature to equal the observed ignition time, the code will only use the measured surface temperature before the ignition

time when evaluating that part of the overall fitness. The actual ϕ value is specified in column F. There are no restrictions on the range of acceptable ϕ values, except that it doesn't make sense to have a negative ϕ value. What matters is the relative values of different ϕ values.

Finally, column G is a small number (ϵ) that prevents the fitness metric from approaching infinity when the model calculation is very close to the experimental data point. See Equation 196 in the Gpyro Technical Reference [1] for further explanation.

14.4. Variables to optimize – GA_Vars worksheet

The model input parameters to be estimated from the available experimental data are specified in the GA_Vars worksheet. Each variable to be optimized is called a gene, and the number of variables to be optimized is specified in cell A1.

Each variable (gene) should be numbered sequentially in column A starting in row 11. In Column B, the name of the worksheet containing the variable to be optimized is specified. The valid choices are indicated in column B, rows 3–8. When optimizing variables from the gyields or hgyields worksheets, the code performs normalization to ensure that the sum of all gyields add to 1 and that the sum of all hgyields add to 0, as is required for mass conservation. However, for this to work correctly, the user must include all nonzero yields as variables to be optimized. For example, if the column of the gyields matrix for a particular condensed phase reaction has four nonzero yields, then these four yields should be specified as variables to be optimized in the GA_Vars worksheet. The homogeneous gaseous yields are similar, except that it is not necessary to specify the yield of reactant 1 because it is always -1.

Column E is the variable (gene) type. The valid variable types for each sheet name are indicated in Column E from rows 3 to 9. For example, if sprops is specified in column B, then the valid variable types are K0Z (thermal conductivity at reference temperature), NKZ (thermal conductivity exponent), and so on. If gyields or hgyields is specified in column B, then it is not necessary to specify a variable type because the code automatically knows that gene must correspond to a species yield.

Two indices are entered in columns D and E (called, very descriptively, Index 1 and Index 2). The role of these two integers depends on the sheet name specified in column B. For example, if the sheet name in column B is sprops, then column C (Index 1) is ISSPEC, the index of solid species as indicated in the sprops sheet. Similarly, if the sheet name in column B is rxns, then column C is IRXN, the index of the reaction (specified in the rxns worksheet). Index 2 does not matter for variables from worksheets sprops or gprops. However, for variables from worksheets gyields and hgyields, Index 1 is the index of the gaseous species, and Index 2 is the reaction number. Thus, if gyields is specified in column B, Index 1 is set to 2, and Index 2 is set to 3, then the gene corresponds to the gaseous yield for the formation of gaseous species 2 from heterogeneous reaction 3.

Columns F and G (MinVal and MaxVal) are the lower and upper bounds of each variable, i.e. the values of $a_{j,\min}$ and $a_{j,\max}$ in Equation 195 of the Gpyro Technical Reference [1]. These should be specified to limit the search space to physically realistic values, i.e. it is known that the thermal conductivity of wood isn't 1000 W/m-K. Column H is a logical parameter indicating whether the values of MinVal and MaxVal are the logarithm of the gene. For example, with a MinVal of 10 and a MaxVal of 12, if log is set to .TRUE. then the code will constrain the search from a minimum value 10^{10} to a maximum value of 10^{12} ; but if log is set to .FALSE., then the code will search for values between 10 and 12.

The mutation probability (see Section 7.2.5 of the Gpyro Technical Reference [1]) is specified in column I. The parameter v_{mut} in Equation 202b of the Gpyro Technical Reference [1] is specified in column J.

The integer in column K provides a convenient means to treat two genes as one parameter for reproduction purposes. For example, the logarithm of the pre-exponential factor and the activation energy are positively correlated, so it may be convenient to form offspring from two parents by applying the same weightings to both the log of the pre-exponential factor and the activation energy. This can be enabled by setting the value in column K to an integer greater than zero. Then, for reproduction purposes, the linear combination applied to the index of the gene specified in column K will also be applied to the current gene.

For solid properties, Index 2 is normally set to 0. However, if Index 2 is set to a solid species index (as specified in sprops) and both MinVal and MaxVal are negative, then the variable is actually the negative of the gene multiplied by the value of the analogous variable of the solid species corresponding to Index 2. This is confusing and is best illustrated by an example: if Index 2 is set to 1, MinVal is set to -1, and MaxVal is set to -0.5, then the gene search space is between 0.5 and 1.0 multiplied by the value of the analogous gene for species 1. The same also applies to the rxns worksheet. Representing the genes in this way is convenient when it is known that one variable must be smaller or larger than another; for example, the thermal conductivity of a char is likely lower than the thermal conductivity of the virgin material from which it was formed, and the activation energy of an oxidative reaction may be lower than the activation energy of an analogous anaerobic reaction.

14.5. Experimental data – 01, 02, 03, etc. worksheets

The experimental data must be pasted into worksheet 01 for case # (or IQE) 1, worksheet 02 for case # (or IQE) 2, and so on. Each ϕ value specified in GA_phi should have an analogous set of experimental data (with the exception of cumulative mass loss, which Gpyro automatically calculates by integrating the mass loss rate). The units for temperature measurements are °C, mass loss rate is g/m²-s, and thickness is m. For a TGA experiment, mass loss rate data should be entered as $1000 \times d(m/m_0)/dt$ where m is the sample mass and m_0 is the initial sample mass. Be sure to express this derivative as d/dt and not d/dT (time derivative instead of temperature derivative).

The experimental data must be entered according to a specific layout, and in an order that corresponds to the order in which the ϕ coefficients are specified in the GA_phi worksheet. Column A is the time axis for the first experimental measurement for which a ϕ value was specified for that IQE in GA_phi. Column B is the corresponding experimental data at each time specified in column A. Column C is blank, and the next set of experimental measurements start in column D, i.e. column D is the time axis for the second experimental measurement, and column E is the analogous experimental data at each time. Column F is blank, and columns G and H would contain the next set of experimental data. Column I is blank, and columns J and K would contain the next set of experimental data, and so on. The order in which the experimental data is specified must correspond to the TYPE order specified in GA_phi. Do not specify experimental data for any cumulative mass loss measurements because this is calculated by integration from the mass loss rate. It is easy to make mistakes in this part of the data input process, and the input routines haven't been rigorously exercised, so be sure to check the output data to make sure that what the code has read in what you intended!

REFERENCES

- [1] Lautenberger, C., “Gpyro – A Generalized Pyrolysis Model for Combustible Solids – Technical Reference Guide,” Version 0.800, May 1, 2014.
- [2] Lautenberger, C.W., “A Generalized Pyrolysis Model for Combustible Solids,” Ph.D. Dissertation, Department of Mechanical Engineering, University of California, Berkeley, Fall 2007.
http://reaxengineering.com/docs/lautenberger_dissertation.pdf
- [3] Lautenberger, C. & Fernandez-Pello, A.C., “Generalized Pyrolysis Model for Combustible Solids,” *Fire Safety Journal* **44** 819-839 (2009).
- [4] Lautenberger, C. & Fernandez-Pello, A.C., “A Model for the Oxidative Pyrolysis of Wood,” *Combustion and Flame* **156** 1503-1513 (2009).
- [5] Dodd, A.B., Lautenberger, C. & Fernandez-Pello, A.C., “Numerical Examination of Two-Dimensional Smolder Structure in Polyurethane Foam,” *Proceedings of the Combustion Institute* **32**: 2497-2504 (2009).
- [6] Lautenberger, C., Kim, E., Dembsey, N. & Fernandez-Pello, C., “The Role of Decomposition Kinetics in Pyrolysis Modeling – Application to a Fire Retardant Polyester Composite,” *Fire Safety Science* **9**: 1201-1212 (2008).
- [7] Lautenberger, C., Rein, G. & Fernandez-Pello, A.C., “Application of a Genetic Algorithm to Estimate Material Properties for Fire Modeling from Bench-Scale Fire Test Data,” *Fire Safety Journal* **41** 204-214 (2006)
- [8] Lautenberger, C. & Fernandez-Pello, C., “Optimization Algorithms for Material Pyrolysis Property Estimation,” *Fire Safety Science* **10** 751-764 (2011).
- [9] Lautenberger, C., “Gpyro3D: A Three Dimensional Generalized Pyrolysis Model,” accepted for publication in *Fire Safety Science* (IAFSS Symposium), 2014.
- [10] McGrattan, K., Hostikka, S., McDermott, R., Floyd, J., Weinschenk, C., and Overholt, K., “Fire Dynamics Simulator Technical Reference Guide Volume 1: Mathematical Model,” NIST Special Publication 1018, Sixth Edition, National Institute of Standards and Technology, Building and Fire Research Laboratory, Gaithersburg, MD, 2015. SVN 22343.

APPENDIX A. NOMENCLATURE

<u>Letters</u>	Description	Units
c_p	Solid heat capacity	
c_{pg}	Gas heat capacity	
d_p	Mean pore diameter	m
D	Mass diffusivity	$\text{m}^2.\text{s}^{-1}$
$\mathfrak{D}$	Deformation	-
E	Activation Energy	$\text{kJ}.\text{mol}^{-1}$
h_c	Convective heat transfer coefficient	$\text{W}.\text{m}^{-2}.\text{K}^{-1}$
k	Heat conductivity	$\text{W}.\text{m}^{-1}.\text{K}^{-1}$
K	Permeability	m^2
m	Mass	kg
M	Molecular weight	$\text{g}.\text{mol}^{-1}$
P	Pressure	Pa
r	Reaction rate	$\text{kg}.\text{m}^{-3}.\text{s}^{-1}$
R	Universal gas constant	$\text{J}.\text{K}^{-1}.\text{mol}^{-1}$
T	Temperature	K
q_{rerad}	Thermal reradiation of the surface	$\text{W}.\text{m}^{-2}$
V	Volume	m^3
X	Volume fraction	m^3/m^3
y	species yield	
Y	Mass fraction	kg/kg
Z	pre-exponential factor	s^{-1}
<u>Greek symbols</u>		
α	Thermal diffusivity	$\text{m}^2.\text{s}^{-1}$
α	Conversion Rate	-
γ	Pore radiation coefficient	m^{-1}
ΔH	Enthalpy of reaction	
Δt	Temporal time step	s
Δx	Spatial step in x-direction	m
Δy	Spatial step in y-direction	m
Δz	Spatial step in z-direction	m
ϵ	Emissivity of the surface	-
θ	Stoichiometric coefficient	$\text{kg}.\text{kg}^{-1}$
ρ	Solid bulk density	$\text{kg}.\text{m}^{-3}$
ρ_s	Solid skeleton density	$\text{kg}.\text{m}^{-3}$
ρ_g	Gas density	$\text{kg}.\text{m}^{-3}$
σ	Stefan–Boltzmann constant	$\text{W}.\text{m}^{-2}.\text{K}^{-4}$
ν	Gas kinematic viscosity	$\text{m}^2.\text{s}^{-1}$

χ	Deformation factor	
ψ	Porosity	-
$\dot{\omega}'''$	Volumetric reaction rate	kg.m ⁻³ .s ⁻¹
<u>Subscripts</u>		
A	Quantity related to solid species A	
B	Quantity related to solid species B	
d	Destruction	
f	Formation	
g	Gas	
i	Quantity related to the solid species of index i	
inf	Quantity at infinity (outside boundary condition)	
j	Quantity related to the gas species of index j	
k	Quantity related to the condensed-phase reaction of index k	
l	Quantity related to the gas-phase reaction of index l	
net	Net production (formation - destruction)	
s	Solid	
sol	Contribution due to the solid phase	
$surf$	Quantity at the surface of a boundary condition	
vol	Contribution due to the volatile phase	
x	Quantity in the x direction	
y	Quantity in the y direction	
z	Quantity in the y direction	

APPENDIX B. NAMELIST GROUP NAMES AND AVAILABLE KEYWORDS.

Table 6: &GPYRO_GENERAL keywords (general parameters)

Keyword	Type	Description	Units	Default
DT0	Real	Section 5.1	s	
TAMB	Real	Section 4.2		
TREF	Real	Section 9.2.1	s	
P0	Real	Section 4.3		
GX	Real	Section 4.3		0
GY	Real	Section 4.3	m.s ⁻²	0
GZ	Real	Section 4.3	m.s ⁻²	0
VHLC	Real	Section 4.2	m.s ⁻²	0
SOVE_GAS_YJ	Integer	Section 13.3.14.4		.FALSE.
SOLVE_GAS_ENERGY	Logical	Section 4.5		.FALSE.
SOLVE_PRESSURE	Logical	Section 4.3		.FALSE.
CONVENTIONAL_RXN_ORDER	Logical	Section 13.3.111.4.3		.FALSE.
BLOWING	Logical	Section 8.2.2		.FALSE.
CONSTANT_DHVOL	Logical	Section 11.5		.TRUE.
QSG_MODEL	String	Section 4.2		'thermal_equilibrium'

Table 7: &GPYRO_CASES keywords (cases to run)

Keyword	Type	Description	Units	Default
NCASES	Integer	Section 5.1		1
IMESH (ICASE)	Integer	Section 5.1		1
TSTOP (ICASE)	Real	Section 5.1	s	
ZEROD (I)	Logical	Section 5.2		F
BETA (I)	Real	Section 5.2	°C/min	

Table 8: &GPYRO_GEOM keywords (geometry specification)

Keyword	Type	Description	Units	Default
NMESH	Integer	Section 6.0		1
ZDIM (IMESH)	Real	Section 6.1	m	
XDIM (IMESH)	Real	Section 6.1	m	1
YDIM (IMESH)	Real	Section 6.1	m	1
NCELLZ (IMESH)	Integer	Section 6.1		
NCELLX (IMESH)	Integer	Section 6.1		1
NCELLY (IMESH)	Integer	Section 6.1		1
DEFAULT_IC (IMESH)	Integer	Section 6.4		
DEFAULT_SURF_IDX (IMESH, 1:6)	Integer	Section 6.4		

NOBST	Integer	Section 6.3		0
IMESH (IOBST)	Integer	Section 6.3		1
Z1 (IOBST)	Real	Section 6.3	m	
Z2 (IOBST)	Real	Section 6.3	m	
X1 (IOBST)	Real	Section 6.3	m	
X2 (IOBST)	Real	Section 6.3	m	
Y1 (IOBST)	Real	Section 6.3	m	
Y2 (IOBST)	Real	Section 6.3	m	
ICNUM (IOBST)	Integer	Section 6.4		DEFAULT_IC
SURF_IDX2D (IOBST, 1:6)	Integer	Section 6.4		DEFAULT_SURF_IDX

Table 9: &GPYRO IC keywords (initial condition specification).

Keyword	Type	Description	Units	Default
NIC	Integer	Section 7.0		1
TMP_INITIAL (IC)	Real	Section 7.0	K	300
TMPG_INITIAL (IC)	Real	Section 7.0	K	300
P_INITIAL (IC)	Real	Section 7.0	Pa	101300
YI0 (IC, ISPEC)	Real	Section 7.0	kg/kg	0
YJ0 (IC, IGSPEC)	Real	Section 7.0	kg/kg	0

Table 10: &GPYRO_ALLBC keywords (boundary condition specification).

Keyword	Type	Description	Units	Default
NSURF_IDX	Integer	Section 8.1		0
SURF_IDX (IBC)	Integer	Section 8.1		
T (IBC)	Real	Section 8.1	s	0
TFIXED (IBC)	Real	Section 8.2.1	K	-1
HC (IBC)	Real	Section 8.2.2	W.m ⁻² .K ⁻¹	10
TINF (IBC)	Real	Section 8.2.2	K	300
QE (IBC)	Real	Section 8.2.2	W.m ⁻²	0
RERADIATION (IBC)	Logical	Section 8.2.2		T
TFIXEDG (IBC)	Real	Section 8.3	K	-1
HCG (IBC)	Real	Section 8.3	W.m ⁻¹ .K ⁻¹	10
TINFG (IBC)	Real	Section 8.3	K	300
QE (IBC)	Real	Section 8.3	W.m ⁻²	0
MDOTPP (IBC)	Real	Section 8.4	g.m ⁻² .s ⁻¹	0
PRES (IBC)	Real	Section 8.4.2	Pa	
YJINF (IBC, IGSPEC)	Real	Section 8.5	Kg/kg	0
HM (IBC)	Real	Section 8.5	kg.m ⁻² .s ⁻¹	

Table 11: &GPYRO_SPROPS keywords (solid properties).

Keyword	Type	Description	Units	Default
NSSPEC	Integer	Section 9.1		

NAME (IDM)	String	Section 9.1		
R0 (IDM)	Real	Section 9.2.3 and 11.2	kg.m ⁻³	
NR (IDM)	Real	Section 9.2.3	-	0
K0Z (IDM)	Real	Section 9.2.1	W.m ⁻¹ .K ⁻¹	
NKZ (IDM)	Real	Section 9.2.1	-	0
K0X (IDM)	Real	Section 9.2.1	W.m ⁻¹ .K ⁻¹	K0Z (IDM)
NKX (IDM)	Real	Section 9.2.1	-	NKZ (IDM)
K0Y (IDM)	Real	Section 9.2.1	W.m ⁻¹ .K ⁻¹	K0Z (IDM)
NKY (IDM)	Real	Section 9.2.1	-	NKZ (IDM)
GAMMA (IDM)	Real	Section 9.2.1	m	0
C0 (IDM)	Real	Section 9.2.2	J.kg ⁻¹ .K ⁻¹	
NC (IDM)	Real	Section 9.2.2	-	0
EMIS (IDM)	Real	Section 9.2.4	-	
KAPPA (IDM)	Real	Section 9.2.4	m ⁻¹	9.10 ⁹
TMELT (IDM)	Real	Section 9.2.2	K	9.10 ⁹
DHMELT (IDM)	Real	Section 9.2.2	J.kg ⁻¹	0
SIGMA2MELT (IDM)	Real	Section 9.2.2	K ²	1
PERMZ (IDM)	Real	Section 9.3.2	m ²	
PERMX (IDM)	Real	Section 9.3.2	m ²	PERMZ (IDM)
PERMY (IDM)	Real	Section 9.3.2	m ²	PERMZ (IDM)
RS0 (IDM)	Real	Section 9.3.1	kg.m ⁻³	R0 (IDM)
PORE_DIAMETER (IDM)	Real	Section 9.3.3	m	

Table 12: &GPYRO GPROPS keywords (gas properties).

Keyword	Type	Description	Units	Default
NGSPEC	Integer	Section 10.1		1
I02	Integer	Section 10.2 and 10.5		
IBG	Integer	Section 10.5		
CPG	Real	Section 10.3	J.kg ⁻¹ .K ⁻¹	1000
NAME (IGAS)	String	Section 10.1		
M (IGAS)	Real	Section 10.4	g.mol ⁻¹	
SIGMA (IGAS)	Real	Section 10.5	Å	
EPSOK (IGAS)	Real	Section 10.5	K	

Table 13: &GPYRO RXNS keywords (condensed phase reaction properties).

Keyword	Type	Description	Units	Default
NRXNS	Integer	Section 11.1		0
CFROM (IRXNS)	String	Section 11.1		
CTO (IRXNS)	String	Section 11.1		
CHI (IRXNS)	Real	Section 11.2	-	1
Z (IRXNS)	Real	Section 11.4.2	s ⁻¹	
E (IRXNS)	Real	Section 11.4.2	kJ.mol ⁻¹	
IKINETICMODEL (IRXNS)	Integer	Section 11.4.3 and Table 3		0

IO2TYPE (IRXNS)	Integer	Section 11.4.3		0
ORDER (IRXNS)	Integer	Section 11.4.3	-	1
ORDERO2 (IRXNS)	Integer	Section 11.4.3	-	0
M (IRXNS)	Real	Section 11.4.3	-	0
KCAT (IRXNS)	Real	Section 11.4.3	-	
ICAT (IRXNS)	Integer	Section 11.4.3		
TCRIT (IRXNS)	Real	Section 11.4.3	K	0
DHV (IRXNS)	Real	Section 11.5	J.kg ⁻¹	0
DHS (IRXNS)	Real	Section 11.5	J.kg ⁻¹	0

Table 14: &GPYRO_GYIELDS keywords (the species-yield matrix GYIELDS).

Keyword	Type	Description	Units	Default
GYIELDS (IGAS, IRXNS)	Real	Section 11.3	-	0

Table 15: &GPYRO_HGYIELDS keywords (the species-yield matrix HGYIELDS).

Keyword	Type	Description	Units	Default
HGYIELDS (IGAS, IGRXNS)	Real	Section 12.1	kg	0

Table 16: &GPYRO_HGRXNS keywords (gas phase reaction properties).

Keyword	Type	Description	Units	Default
NHGRXNS	Integer	Section 12.1		0
CREACTANT1 (IGRXNS)	String	Section 12.1		
CREACTANT2 (IGRXNS)	String	Section 12.1		
P (IGRXNS)	Real	Section 12.2	-	1
Q (IGRXNS)	Real	Section 12.2	-	1
B (IGRXNS)	Real	Section 12.2	-	0
Z (IGRXNS)	Real	Section 12.2	s ⁻¹	
E (IGRXNS)	Real	Section 12.2	kJ.mol ⁻¹	
DH (IGRXNS)	Real	Section 12.3	J.kg ⁻¹	0

Table 17: &GPYRO_OUTPUT keywords (output quantities).

Keyword	Type	Description	Units	Default
CASENAME	String	Section 13.0		'gpyro'
N_POINT_QUANTITIES	Integer	Section 13.2.1		0
DTDUMP_POINT	Real	Section 13.2.1	s	
POINT_QUANTITY (I)	String	Section 13.2.1		
POINT_QUANTITY_INDEX (I)	Integer	Section 13.2.1		
POINT_Z (I)	Real	Section 13.2.1	m	
POINT_X (I)	Real	Section 13.2.1	m	
POINT_Y (I)	Real	Section 13.2.1	m	

POINT_IMESH(I)	Integer	Section 13.2.1		1
N_PROFILE_QUANTITIES	Integer	Section 13.3.1		0
DTDUMP_PROFILE	Real	Section 13.3.1	s	
PROFILE_QUANTITY(J)	String	Section 13.3.1		
PROFILE_QUANTITY_INDEX(J)	Integer	Section 13.3.1		
PROFILE_DIRECTION(J)	String	Section 13.3.1		
PROFILE_COORD1(J)	Real	Section 13.3.1	m	
PROFILE_COORD2(J)	Real	Section 13.3.1	m	
PROFILE_IMESH(J)	Integer	Section 13.3.1		1
PROFILE_ISKIP(J)	Integer	Section 13.3.1		1
N_SMOKEVIEW_QUANTITIES	Integer	Section 13.4.1		0
DTDUMP_SMOKEVIEW	Real	Section 13.4.1	s	
SMOKEVIEW_QUANTITY(K)	String	Section 13.4.1		
SMOKEVIEW_QUANTITY_INDEX(K)	Integer	Section 13.4.1		
SMOKEVIEW_PLANE(K)	String	Section 13.4.1		
SMOKEVIEW_LOCATION(K)	Real	Section 13.4.1	m	
TMP_REDUCED_DTDUMP	Real	Section 13.5	K	9. 10 ⁹
REDUCED_DTDUMP	Real	Section 13.5	s	
DUMP_DETAILED_CONVERGENCE	Logical	Section 13.6.3		F

Table 18: &GA_GENINPUT keywords (property estimation general/miscellaneous).

Keyword	Default value	Units	Description
NINDIV	250		Population size (#of individuals)
NGEN	100		Number of generations (evolution steps)
MAXCOPIES	6		Maximum number of copies - target sel rate
LINUX	.TRUE.		Is this run under Linux?
SIMULATED_EXPERIMENTAL_DATA	.FALSE.		Generate synthetic experimental data?
RESTART	.FALSE.		Restart from file restart.restart?
FITMIN	0.0		Set very bad fitness to this
FITCLIP	0.0		If fitness is <= this or NaN, set fitness to this
FITEXPONENT	2.0		Raise fitness value to this exponent
WHOLEGENEFRACTION	0.5		Fraction of genes that are not linear combins.
BRUTE_FORCE	.FALSE.		Use brute force (no genetic algorithm?)
KILL_NONCONVERGED_SOLNS	.TRUE.		Kill solutions that don't converge?

APPENDIX C. GAS PROPERTIES

Substance	Molecular Weight M (g/mol)	Lennard–Jones Parameters σ (Å)	ϵ/k (K)
<i>Light elements:</i>			
H ₂	2.016	2.915	38.0
He	4.003	2.576	10.2
<i>Noble gases:</i>			
Ne	20.183	2.789	35.7
Ar	39.944	3.418	124.
Kr	83.80	3.498	225.
Xe	131.3	4.055	229.
<i>Simple polyatomic substances:</i>			
Air	28.97	3.617	97.0
N ₂	28.02	3.681	91.5
O ₂	32.00	3.433	113.
CO	28.01	3.590	110.
CO ₂	44.01	3.996	190.
NO	30.01	3.470	119.
N ₂ O	44.02	3.879	220.
SO ₂	64.07	4.290	252.
F ₂	38.00	3.653	112.
Cl ₂	70.91	4.115	357.
Br ₂	159.81	4.268	520.
I ₂	253.82	4.982	550.
<i>Hydrocarbons:</i>			
CH ₄	16.04	3.822	137.
C ₂ H ₂	26.04	4.221	185.
C ₂ H ₄	28.05	4.232	205.
C ₂ H ₆	30.07	4.418	130.
C ₃ H ₈	44.09	5.061	254.
i-C ₄ H ₁₀	58.12	5.341	313.
n-C ₅ H ₁₂	72.15	5.769	343.
n-C ₆ H ₁₄	86.17	5.909	413.
n-C ₈ H ₁₈	114.22	7.451	320.
Cyclohexane	84.16	6.093	324.
C ₆ H ₆	78.11	5.270	440.
<i>Other organic compounds:</i>			
CH ₄	16.04	3.822	137
CH ₃ Cl	50.49	3.375	855.
CH ₂ Cl ₂	84.94	4.759	406.
CHCl ₃	119.39	5.430	327.
CCl ₄	153.82	5.881	327.
C ₂ N ₂	62.04	4.38	339.
COS	60.08	4.13	335.
CS ₂	76.14	4.438	488.

